
分布式文件系统



崔立骁

Slides来自清华大学
存储课程



目录

□ 背景

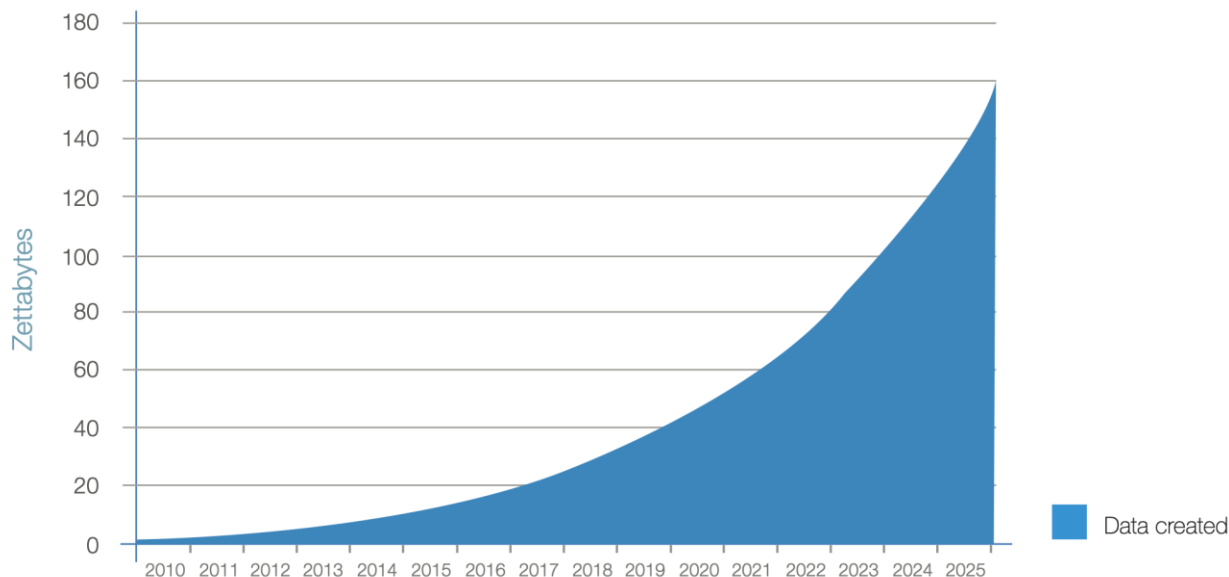
□ 基本架构：GFS

□ 数据分布

□ 元数据组织

数据量日益增大

▣ IDC《Data Age 2025》估计，全球每年产生的数据将从2018年的33ZB增长到175ZB，相当于每天产生491EB的数据

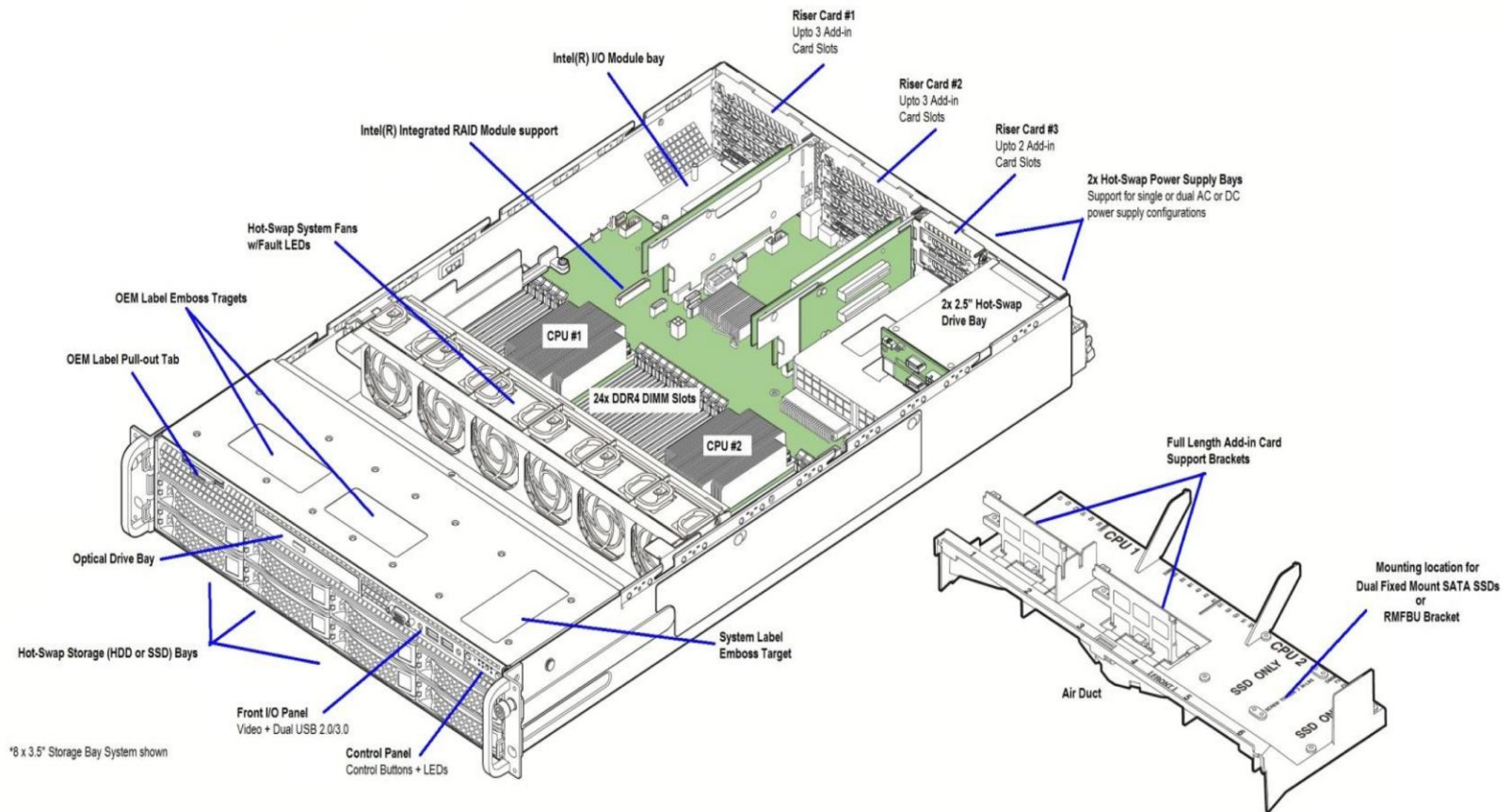


Source: IDC's Data Age 2025 study, sponsored by Seagate, April 2017

▣ 如果把175ZB全部存在DVD光盘中，那么DVD叠加起来的高度将是地球和月球距离的23倍（月地最近距离约39.3万公里），或者绕地球222圈（一圈约为四万公里）

单机难以扩展

❑ 受限于DIMM和PCIe endpoint数量，单台机器的内存和外存不能无限扩展



分布式文件系统

□ 定义

- 分布式文件系统（Distributed File System, DFS）是单机文件系统的分布式扩展，它管理着多台服务器上的存储资源，并允许用户在网络任意节点上像访问本地文件系统一样访问文件

□ 设计目标

- 可扩展性：优雅地横向扩展（scale out）到数百至数万节点
- 访问透明性：用户能够像访问本地文件系统一样访问DFS
- 位置透明性：用户通过全局命名空间访问文件
- 并发透明性：多个用户并发访问文件时，看到的状态是一致的
- 迁移透明性：文件迁移不影响用户访问，保证负载均衡
- 副本透明性：维护数据副本或纠删码，保证数据可靠性
- 故障透明性：容忍节点故障，保证系统可用性
- 异构性：兼容不同的硬件和操作系统平台

分布式文件系统基本架构

□ 客户端

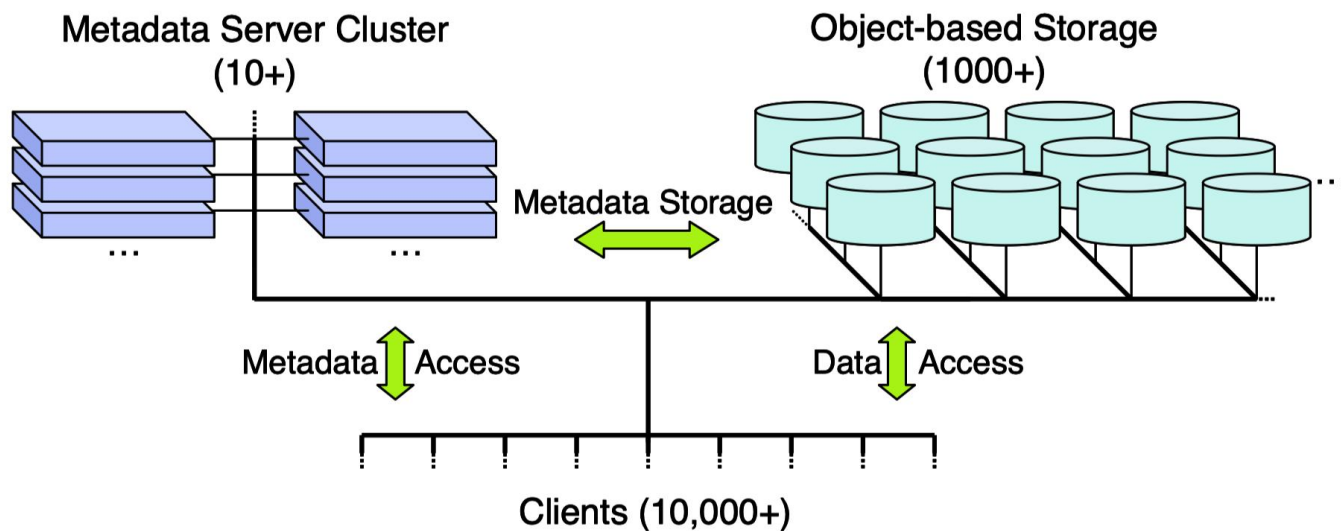
- 向用户提供访问接口

□ 元数据子系统

- 管理命名空间（目录树）和文件元数据（数据映射等）

□ 数据子系统

- 管理文件数据块



文件系统接口

□ POSIX

- 按照POSIX标准实现的VFS接口，可以直接挂载至本地文件系统上。

□ NFS/CIFS

- 采用标准的文件共享访问协议，具有非常好的互操作性，可以直接挂载至本地文件系统上。

□ MPI

- 在POSIX的基础上增加面向高性能计算的多节点进程同步，并行访问的接口。(GlusterFS)

□ FUSE(用户态Posix接口)

- 可挂载，性能较差，多用于原型系统开发

□ 自定义API

- 与特定的应用程序配合使用(GoogleFileSystem)

目录

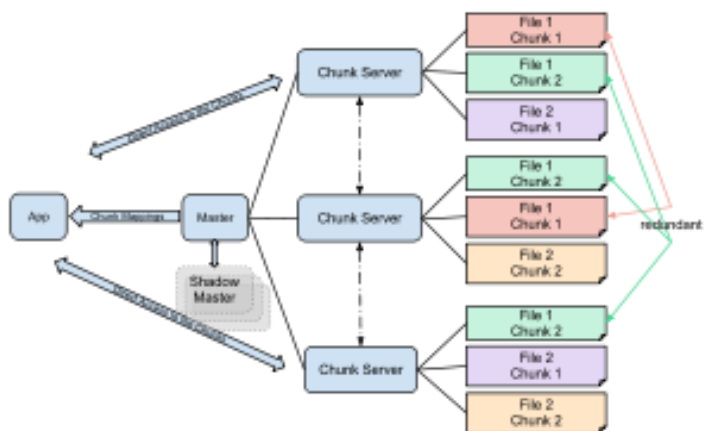
□ 背景

□ 基本架构：GFS

□ 数据分布

□ 总结

Google的三驾马车



Google File System



MapReduce

Google
BigTable



BigTable

GFS设计考量

❑ 出错是常态

- 节点失效
- 硬盘失效
- 网络故障
- 人为操作
- ...

❑ “Therefore, constant monitoring, error detection, fault tolerance, and automatic recovery must be integral to the system.”

GFS设计考量

□ 文件通常是大文件

- 大数据和数据中心的应用，GB级的文件已经非常普遍
- 认为KB级的文件都是小文件
- 分布式文件系统应该为大文件设计

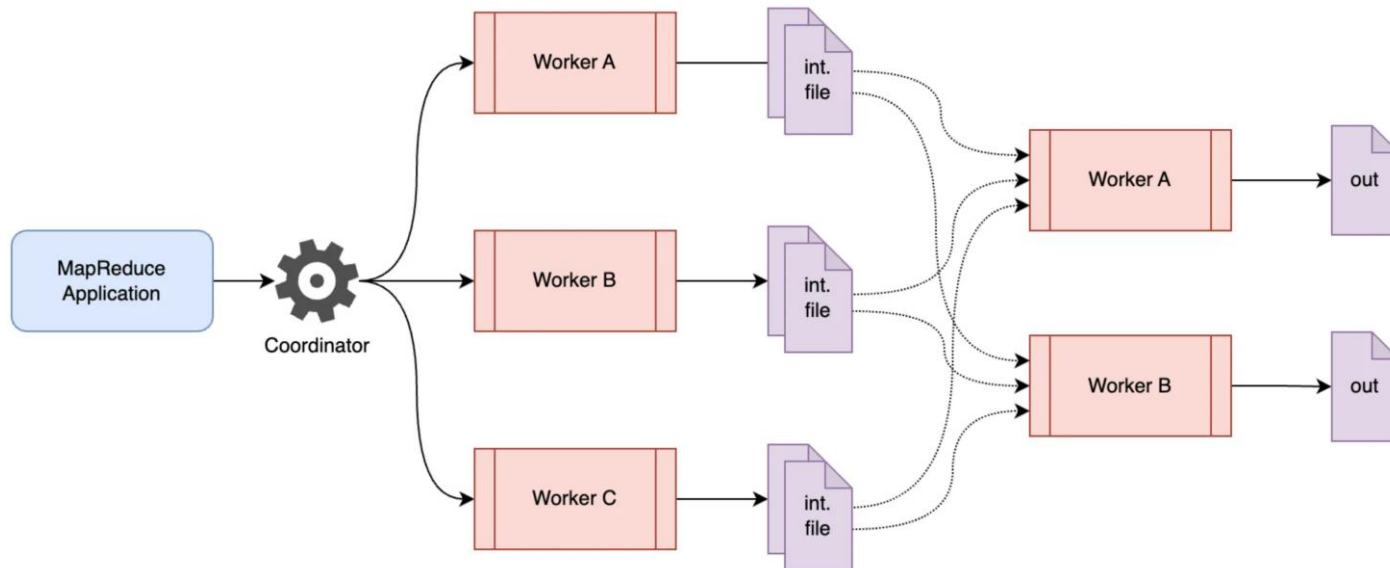
□ “design assumptions and parameters such as I/O operation and block sizes have to be revisited.”

GFS设计考量

❑ 文件操作以append为主

- 少量的原地修改
- 大量的追加写入

❑ “appending becomes the focus of performance optimization and atomicity guarantees, while caching data blocks in the client loses its appeal.”



GFS设计考量

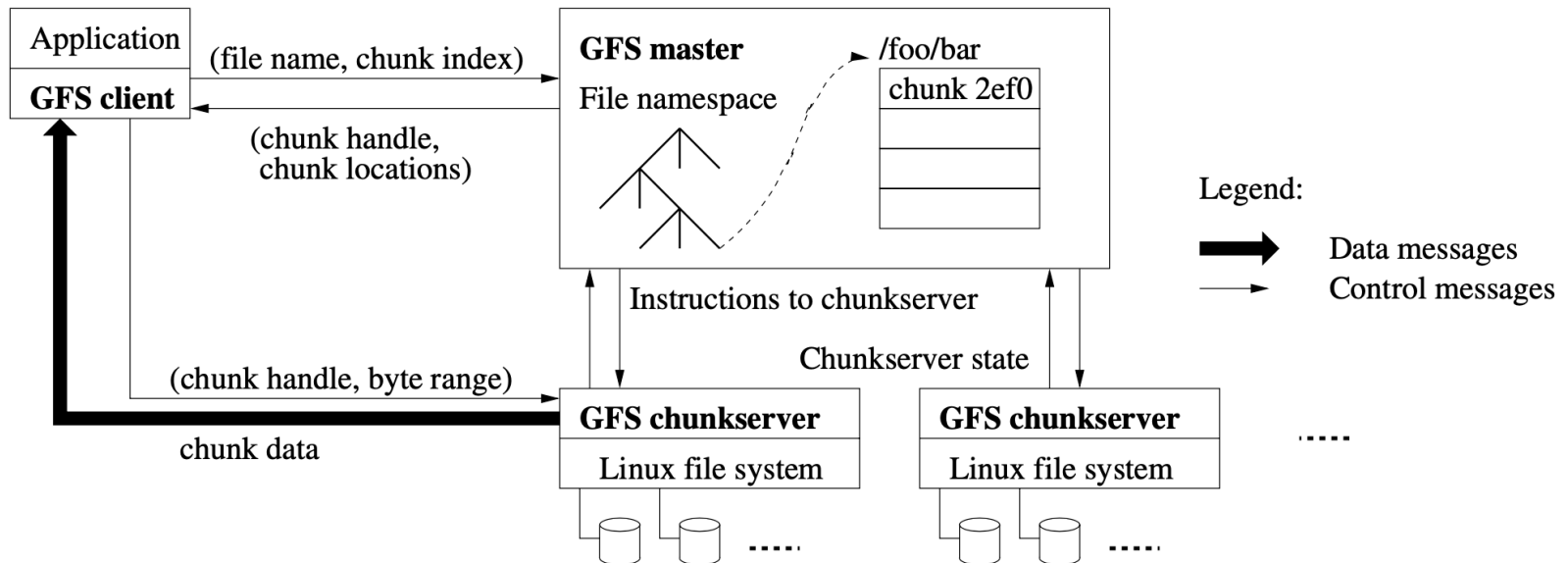
□ 协同设计

- 通过放松一致性语义提高性能
- 但是需要应用修改以匹配

□ “co-designing the applications and the file system API benefits the overall system by increasing our flexibility.

基本架构

- 一个Master节点 - 元数据节点
- 多个Chunkserver节点 - 数据节点



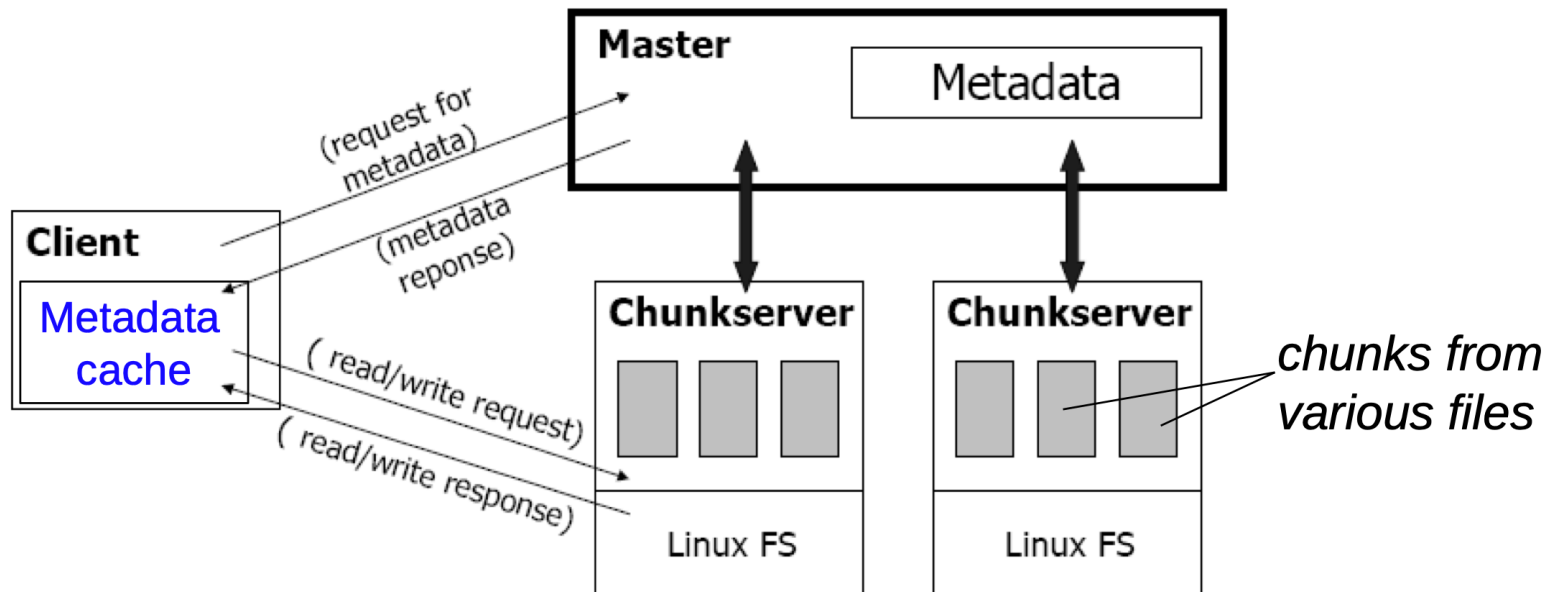
Single Master

- 功能：存储文件目录信息，处理元数据相关操作
 - 目录树
 - 文件和chunk块的映射
 - chunk位置信息
- 优点：单台服务器，简化了设计
- 缺点：性能问题和可用性问题
 - 单台服务器性能有限
 - 崩溃之后整个文件系统无法工作

Single Master

□ 如何缓解潜在的性能问题？

- 元数据路径和数据路径分离，master只处理元数据操作，客户端的文件操作请求先发往master，返回数据在chunkserver的位置信息后，客户端的IO操作直接和chunkserver通信，避免了master位于关键路径上



Single Master

□ 如何缓解潜在的可用性问题？

- 通过冗余来解决可用性问题
- 主从备份，backup node又称为shadow master，master处理的所有元数据操作，会通过log的形式发往shadow master
- 一旦master失效，shadow master通过replay log来恢复元数据，成为新的master

Chunk servers

- ❑ 功能：存储数据内容，服务客户端的读写请求
- ❑ 文件被切分成多个固定大小的条带（chunk），条带分散存储在多台chunk server上，GFS中chunk大小为64MB
- ❑ 优点：大chunk让文件元数据更小
- ❑ 缺点：对小文件来说浪费空间

Chunk servers

□ 数据可用性如何保证？

- 也是通过冗余

- 一个chunk会被存储在台chunk servers上（默认三台）

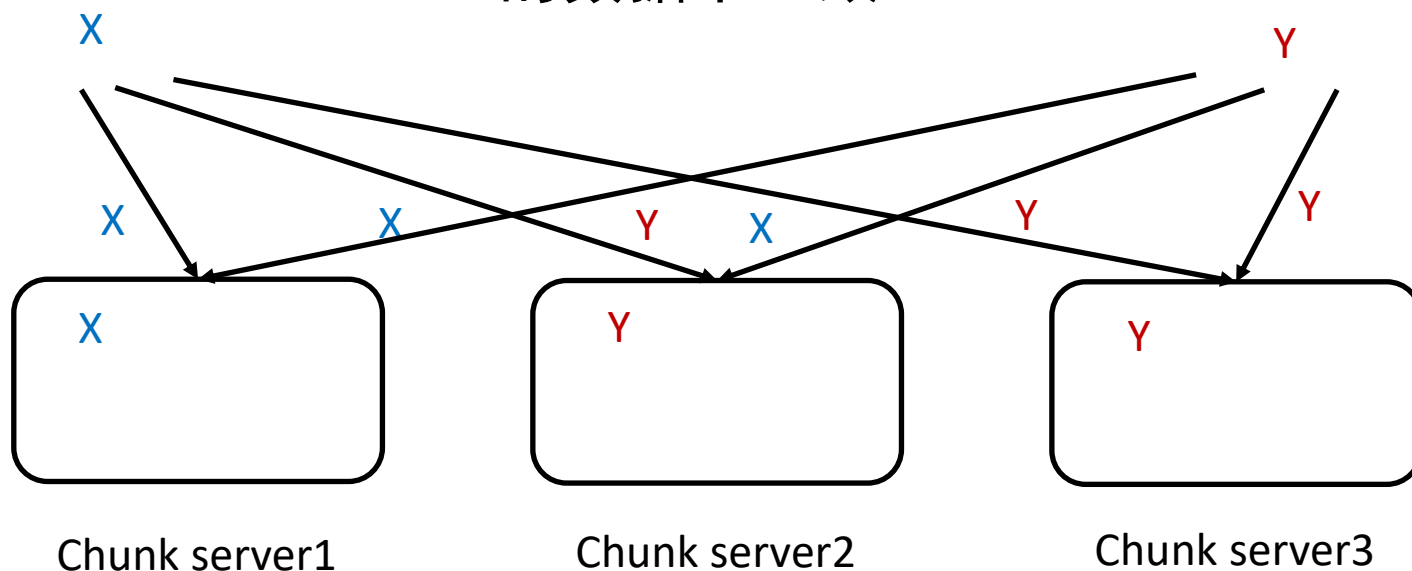
□ 对于读请求，客户端先向master请求chunk的位置信息，包括chunk server的物理位置等，随后和chunk server通信读取数据

□ 如何处理写请求？

Chunk servers

□ 如何处理写请求？

- naive的处理方式会导致数据的不一致
- Case: 对同一个chunk写三副本，由于数据到达的顺序不一致，可能会导致三副本最终数据不一致，例如写X和Y，由于chunkserver1先收到Y再收到X，会导致和chunkserver 2 / 3的数据不一致

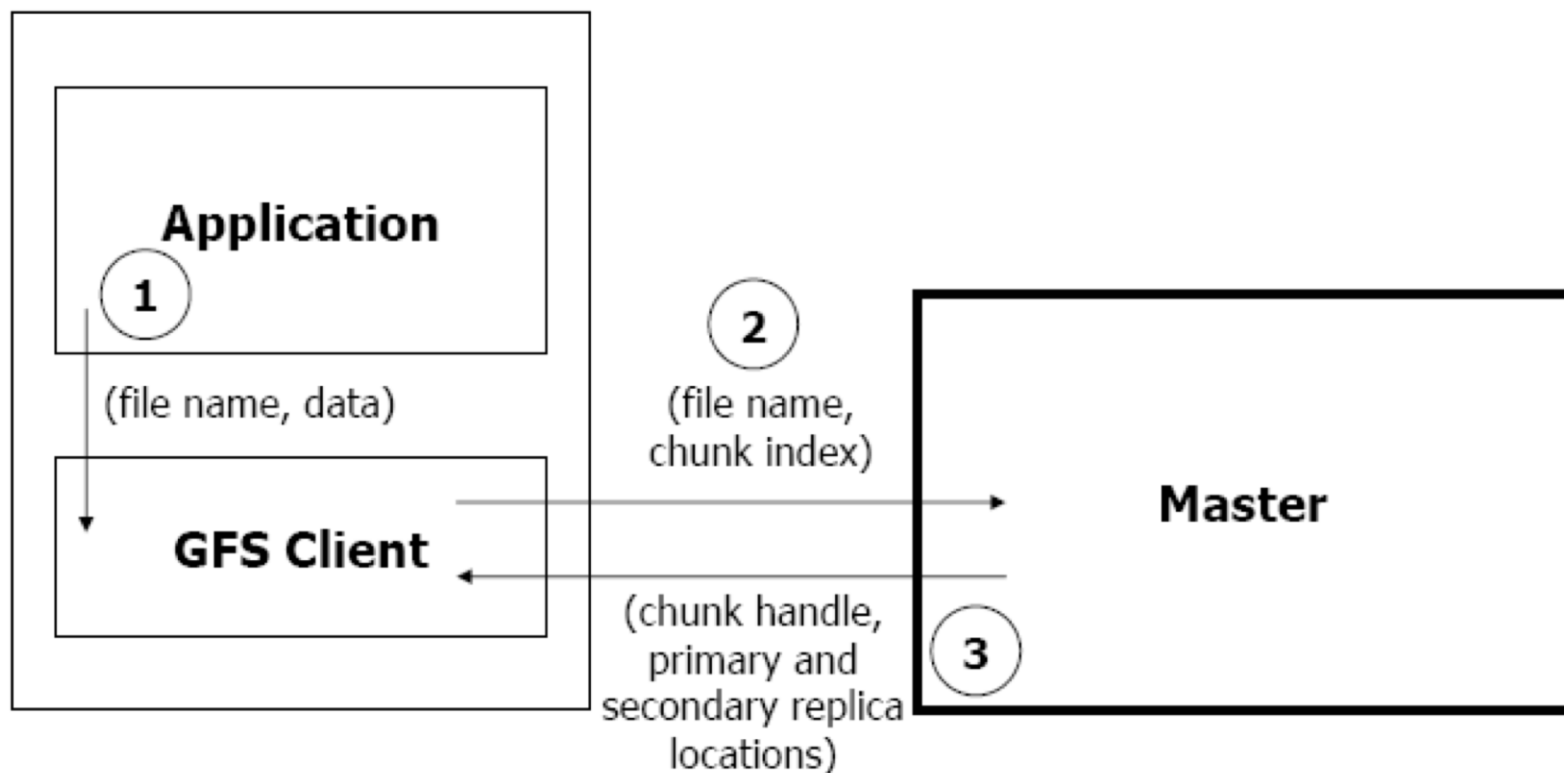


Chunk servers

- 如何解决？
- 定序：需要确定操作的顺序
- 每一个chunk的三副本，由master决定哪个是primary，primary来决定写入的顺序

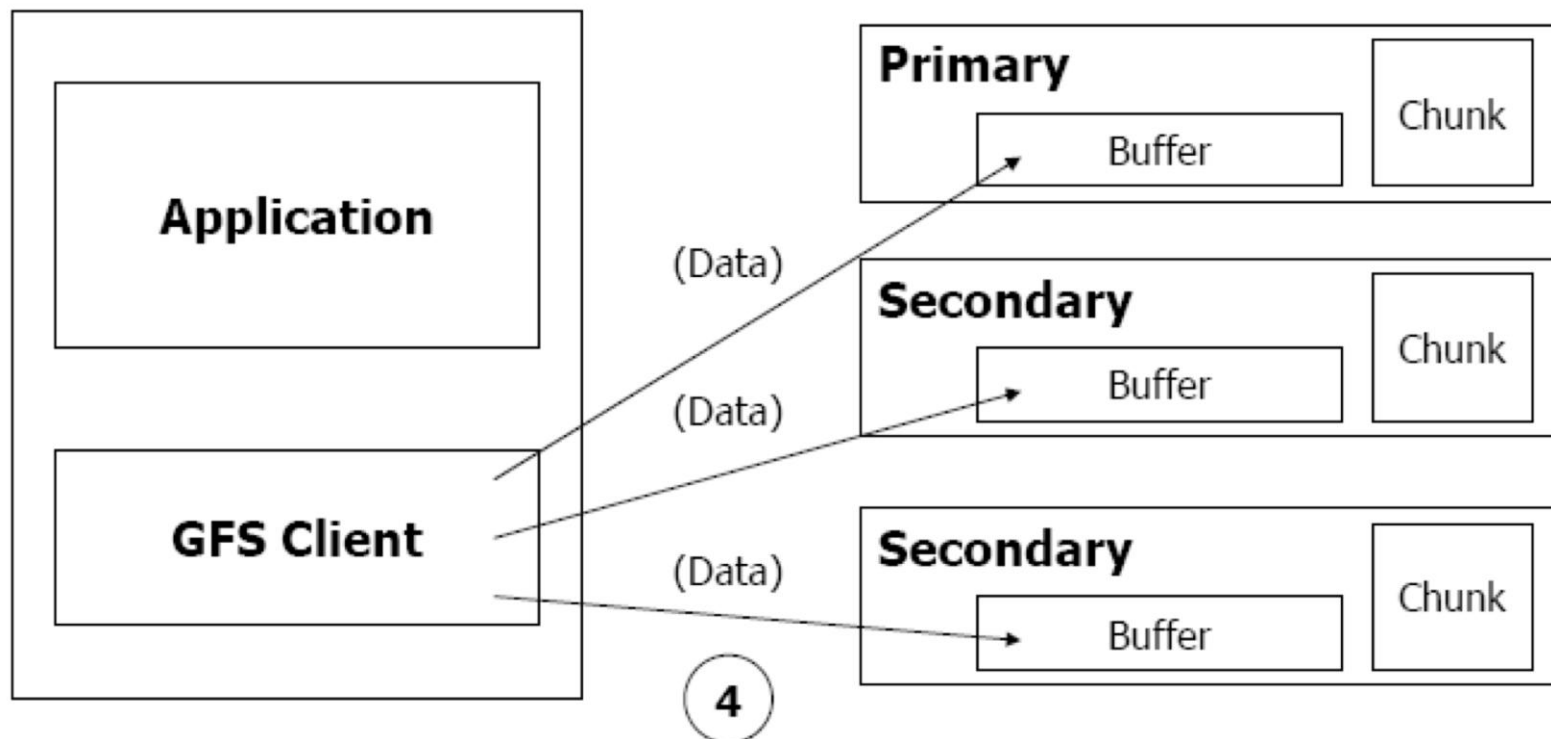
Chunk servers

1. 应用发起一次写操作
2. 客户端向master发送文件名， chunk号等信息
3. master查找元数据， 将chunk位置信息返回客户端



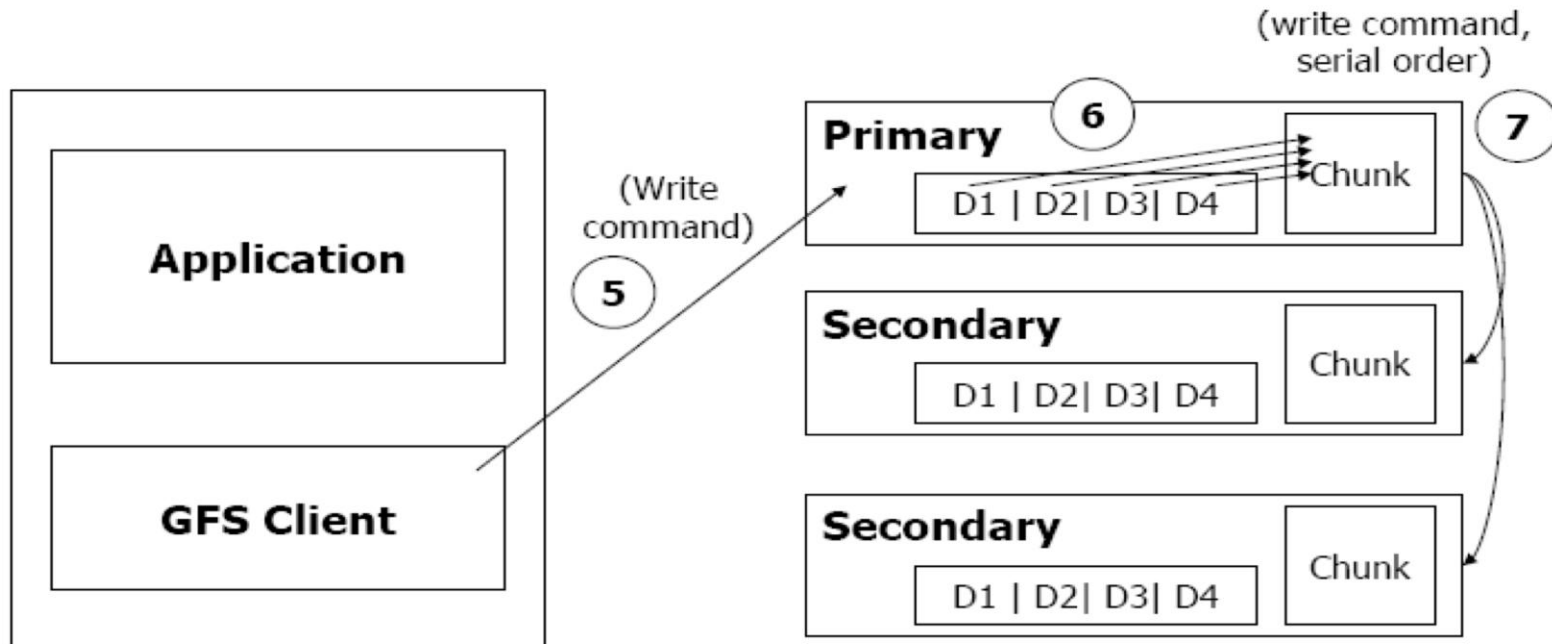
Chunk servers

4. 客户端根据master返回的三副本位置信息广播要写的数据到三台Chunkservers，Chunkserver收到数据后存在buffer中，并不执行写操作



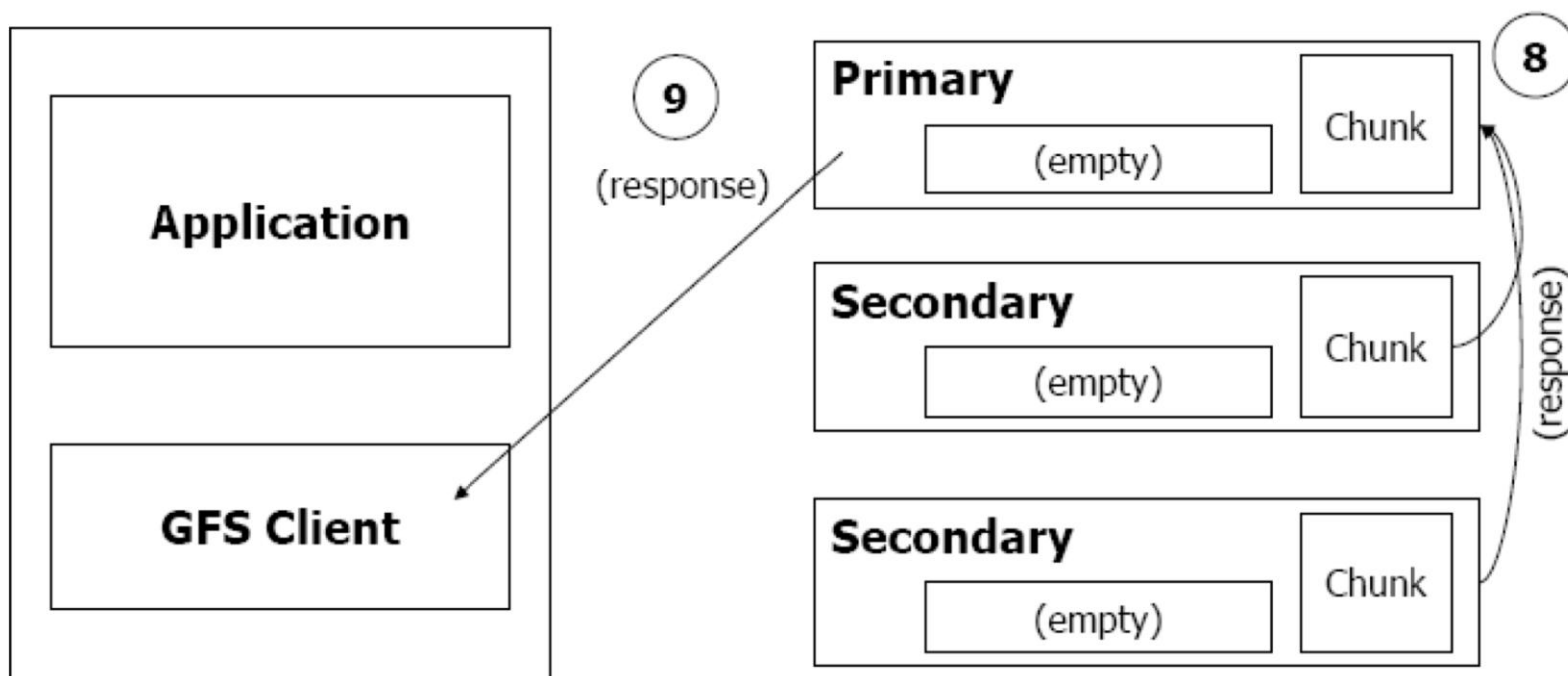
Chunk servers

5. 客户端向primary发送write command
6. Primary给每个缓存的写操作定义顺序，并根据该顺序写入chunk
7. Primary向其他两台服务器广播该顺序



Chunk servers

8. 每个secondary写入chunk之后，完成返回信息
9. Primary等到所有的secondary都返回成功之后，返回客户端



Chunk servers

- 上述协议能保证每个chunk在三副本上保持一致
- 但对于写操作来说，仍然会导致一些不一致
 - Consistent but undefined
- 考虑两个并发的写操作，写XX和写YY
 - 期待的结果：XX或者YY
 - 不期待的结果：XY，YX，或者ZZ
- 上述写协议会让我们得到期待的结果吗？

Chunk servers

□ NO!

□ 无法保证多个chunk之间的顺序

□ 考虑如下情况

- chunk 32MB

- 第一个写64MB，写的内容为X

- 第二个写64MB，写的内容为Y

- 会出现第一个chunk全为X，第二个chunk全为Y的情况

□ 仅仅根据primary来决定一个chunk内的顺序并不足以解决多个chunk的更新顺序问题

Chunk servers

- 如何多个chunk的更新顺序问题
- 引入分布式事务！
 - 基本做法：将操作涉及的所有primary阻塞住（通过锁），写完操作后放行。
 - 性能低！
- GFS为了性能舍弃了分布式事务，让上层应用处理多chunk更新导致的异常

目录

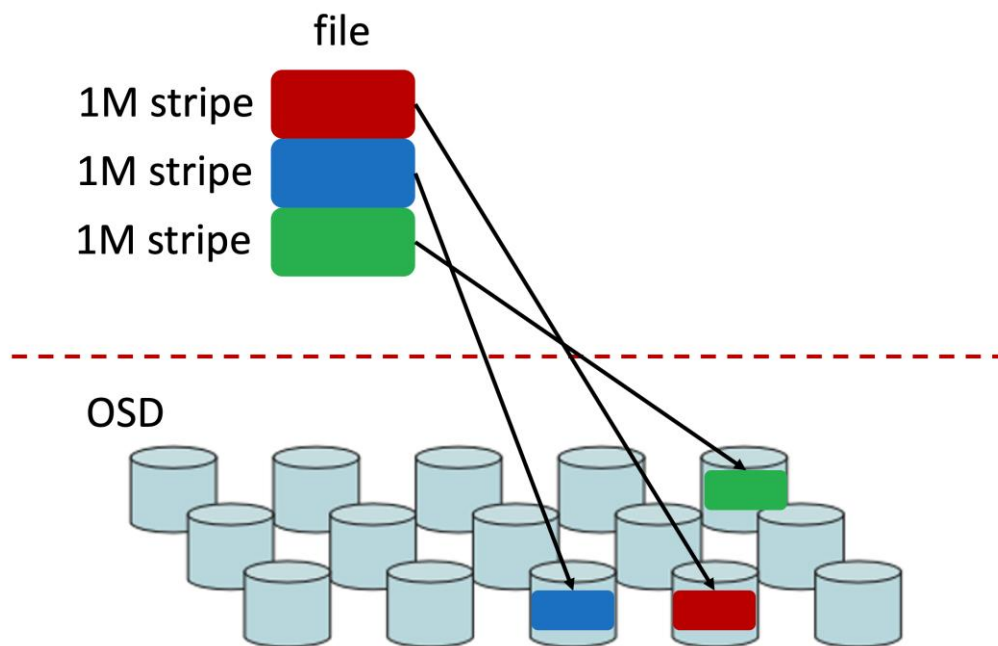
- 背景
- 基本架构：GFS
- 数据分布**
- 元数据组织

数据分布

❑ 分布式文件系统的一个文件，会被切分成多个Chunk，按照某种算法，分布在多台数据存储服务器上

❑ 优点？

❑ 缺点？



数据块分布

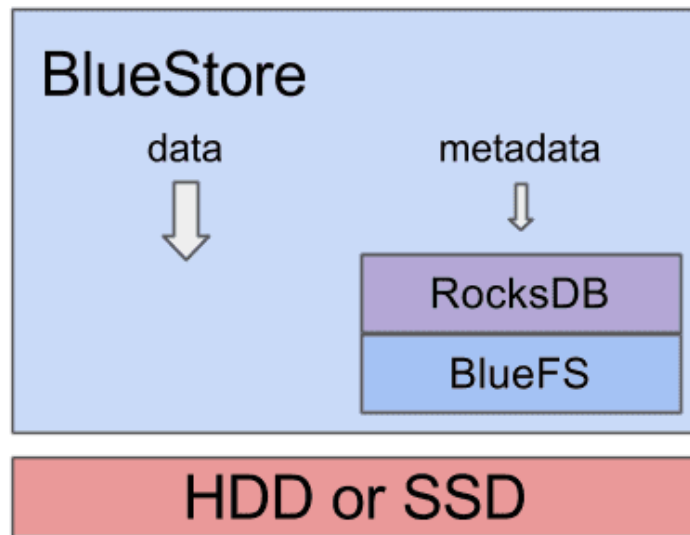
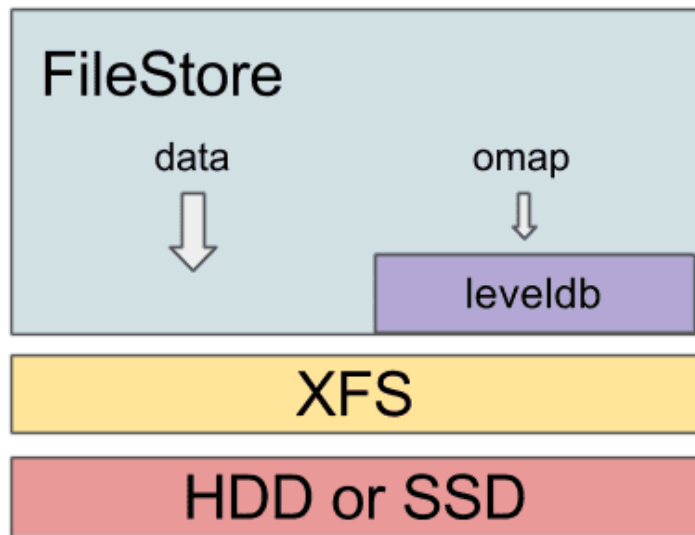
□ 两大类的数据块分布策略

- 去中心化，依赖于分布式的算法决定数据块位置
 - Ceph, DAOS
- 中心化，依赖于元数据服务器分配数据块位置
 - Lustre, BeeGFS

数据块分布

□ 后端存储

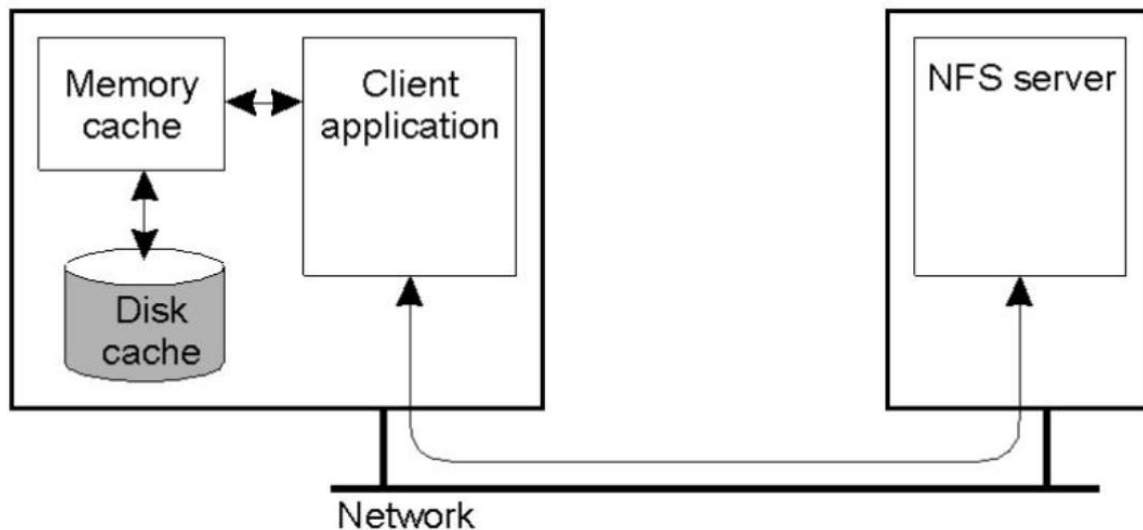
- 基于本地文件系统的存储，接口简单，层次复杂
- 基于裸设备的存储方式，需要提供磁盘的管理，层次简单 (Ceph BlueStore)



数据块分布

□ 数据缓存

- 在客户端缓存数据块
- 好处：高性能、减少网络和后端存储的压力
- 难度：如何维护缓存一致性



数据分布的重要性

□ 数据块的分布决定了持久化和可用性

- 决定多副本的位置，例如GFS中，需要为每个chunk选取三个chunk servers
- 决定了故障域（Failure domain），同一个机架/机房/数据中心的机器往往同时失效，数据块的分布需要考虑多副本位于多个故障域

□ 数据块的分布决定了负载均衡和数据迁移性能

- 决定了多台数据服务器数据是否均衡，从而在服务IO请求时能否负载均衡，发挥出后端数据服务器的并行性能
- 决定了增删机器时数据迁移的数量

经典的数据分布算法

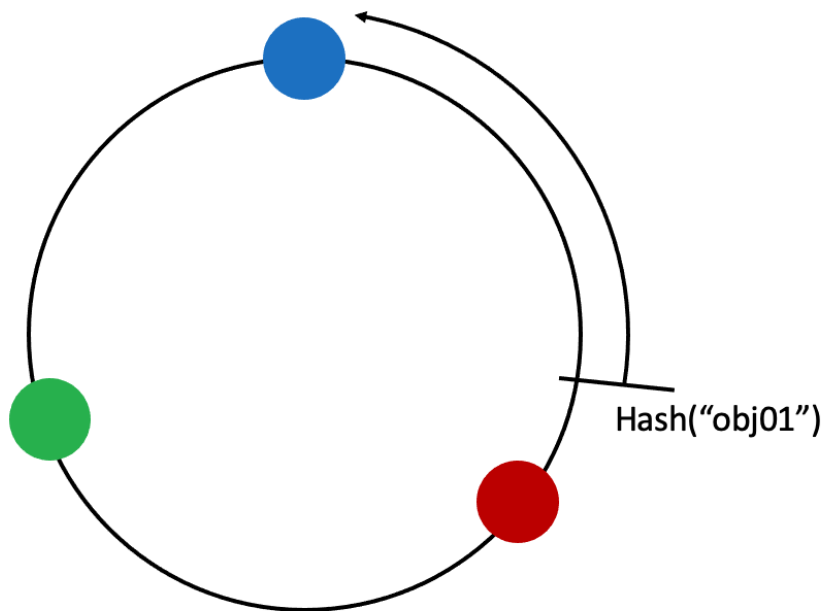
- 一致性hash
- Ceph CRUSH
- 映射表（中心化数据分布）

一致性hash

- 数据分布就是把X分配到某台服务器i上
- 假如我们用X直接取模
 - 服务器编号从 1..k
 - 将X放置于 $i = (X \bmod k)$
- 有什么问题？
 - 数据可能不均衡
- 解决：先HASH
 - 将X放置于 $i = \text{hash}(X) \bmod k$
- 有什么问题？
 - 如果有服务器增加或者减少($k \rightarrow k \pm 1$)?
- 引发数据的大量迁移！！！！

一致性hash

- 一致性hash将存储节点和数据都映射到一个首尾相连的哈希环上
- 将数据块的key通过hash函数计算之后，分配给第一台逆时针（或顺时针）的机器

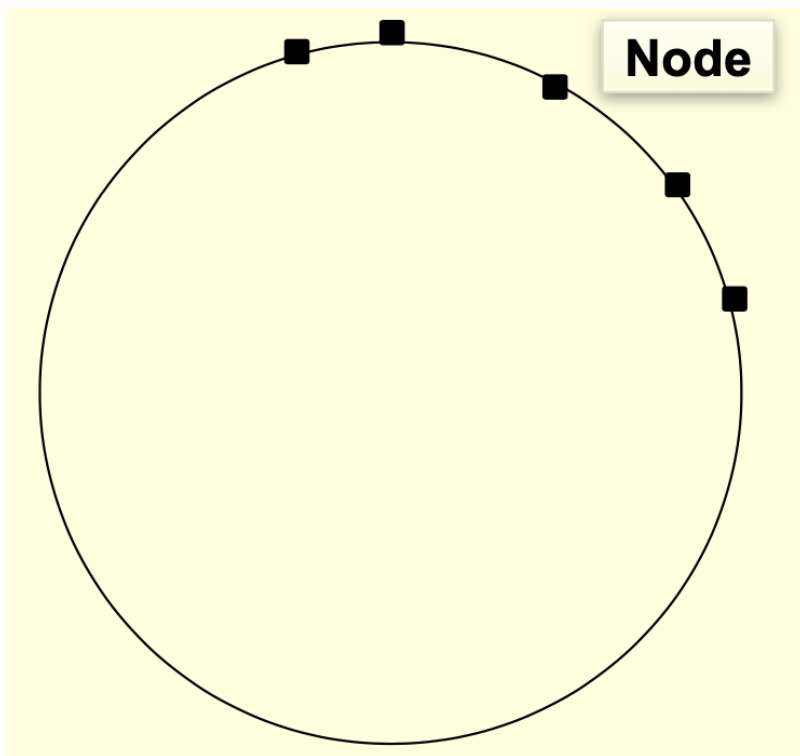


一致性hash

- 如果有节点加入/删除如何处理？
- 需要将前置节点的数据移动到新加入的节点/要删除的节点移动自己的数据到前置节点
- 需要移动的数据为 $O(1/N)$

一致性hash

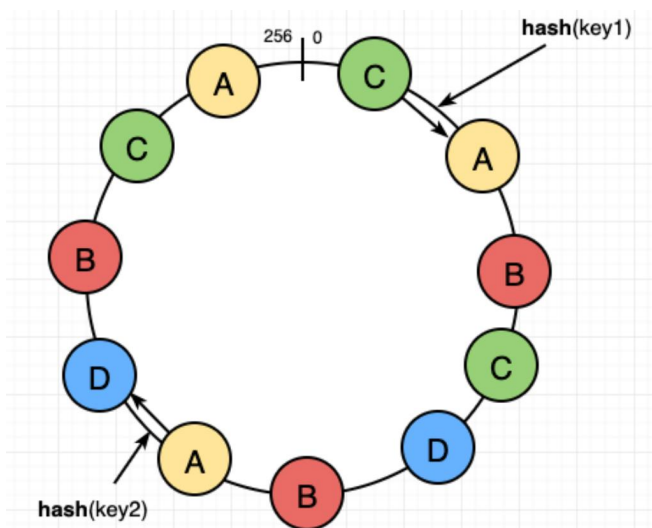
- 数据负载不均衡?
- 少量的数据节点会导致大量数据被分配到某几台服务器上



一致性hash

□ 解决：虚拟节点

- 每一个物理的存储服务器，都被映射到多个虚拟节点，从而在哈希环上有更多的节点，保证了数据的负载均衡
- 虚拟节点的引入并不影响增删节点的数据移动数量

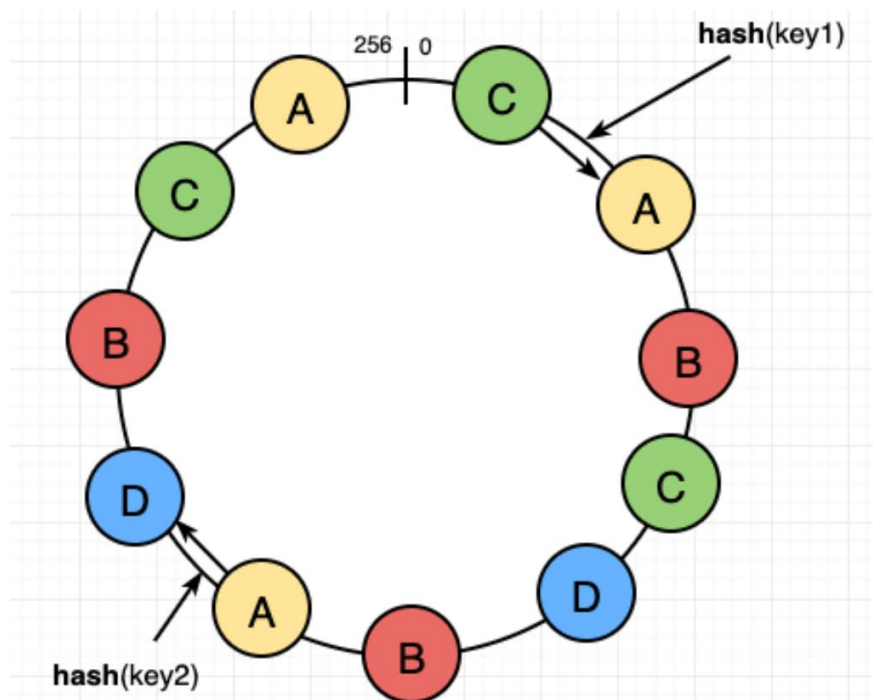


□ 虚拟节点还能处理多台服务器性能不均的问题

- 某些服务器存储性能更强，可以分配更多的虚拟节点，从而有更大的概率分配到数据块

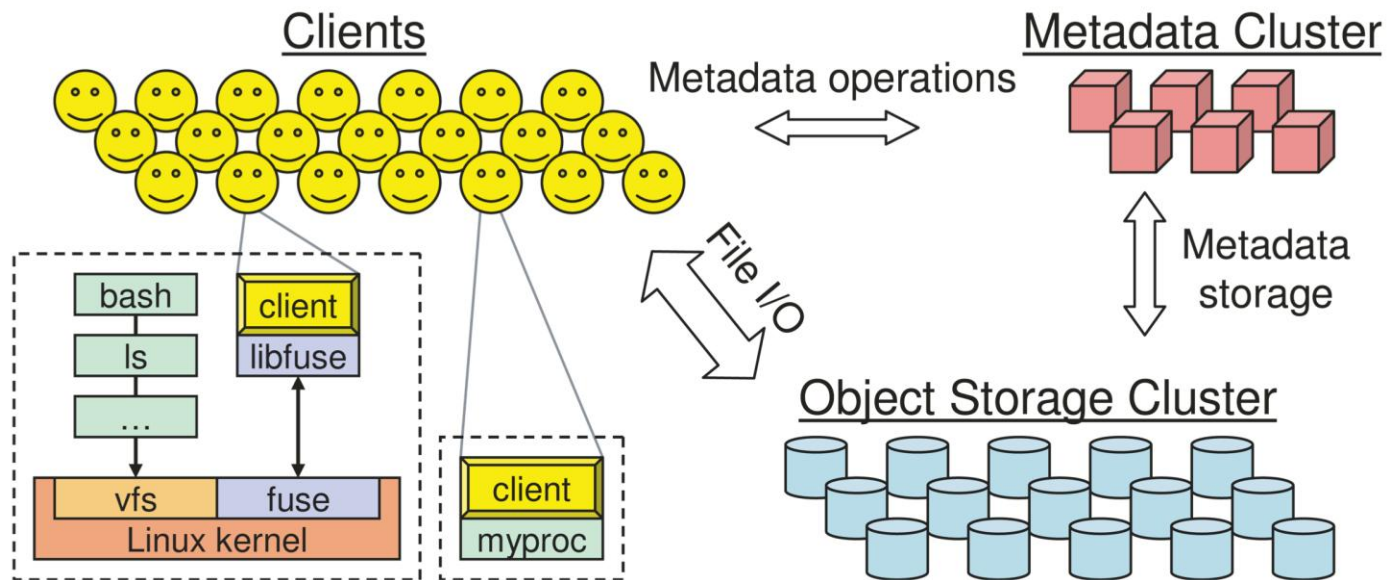
一致性hash

- ❑ 如何支持多副本？
- ❑ 哈希环中连续的n个节点
- ❑ key1被存储在A、B、C，其中A是primary



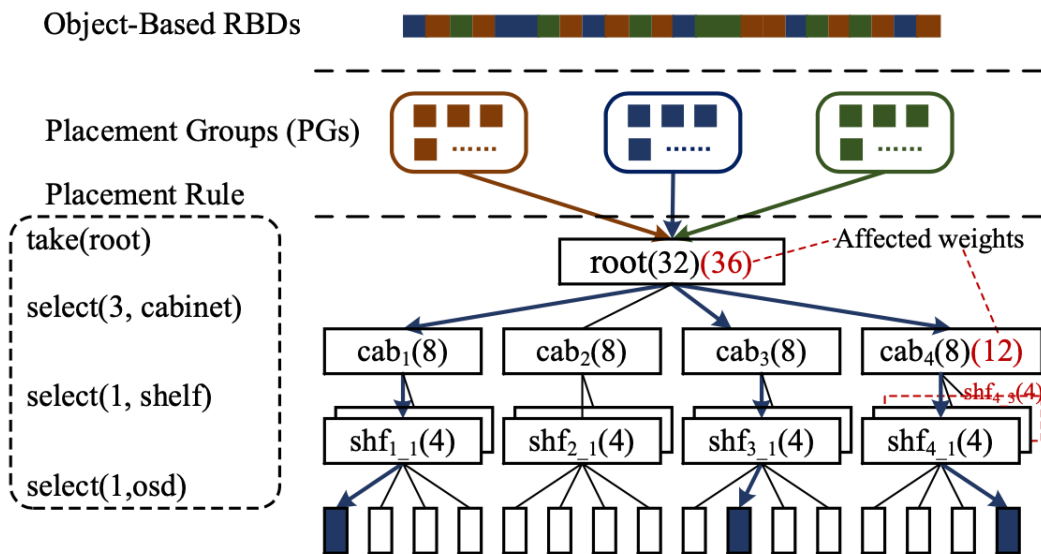
Ceph-CRUSH算法

- ❑ Ceph分为元数据集群和数据集群
- ❑ 与GFS不同的是，数据分布并不存储在元数据集群中，而是用一个分布式算法来决定数据块的位置



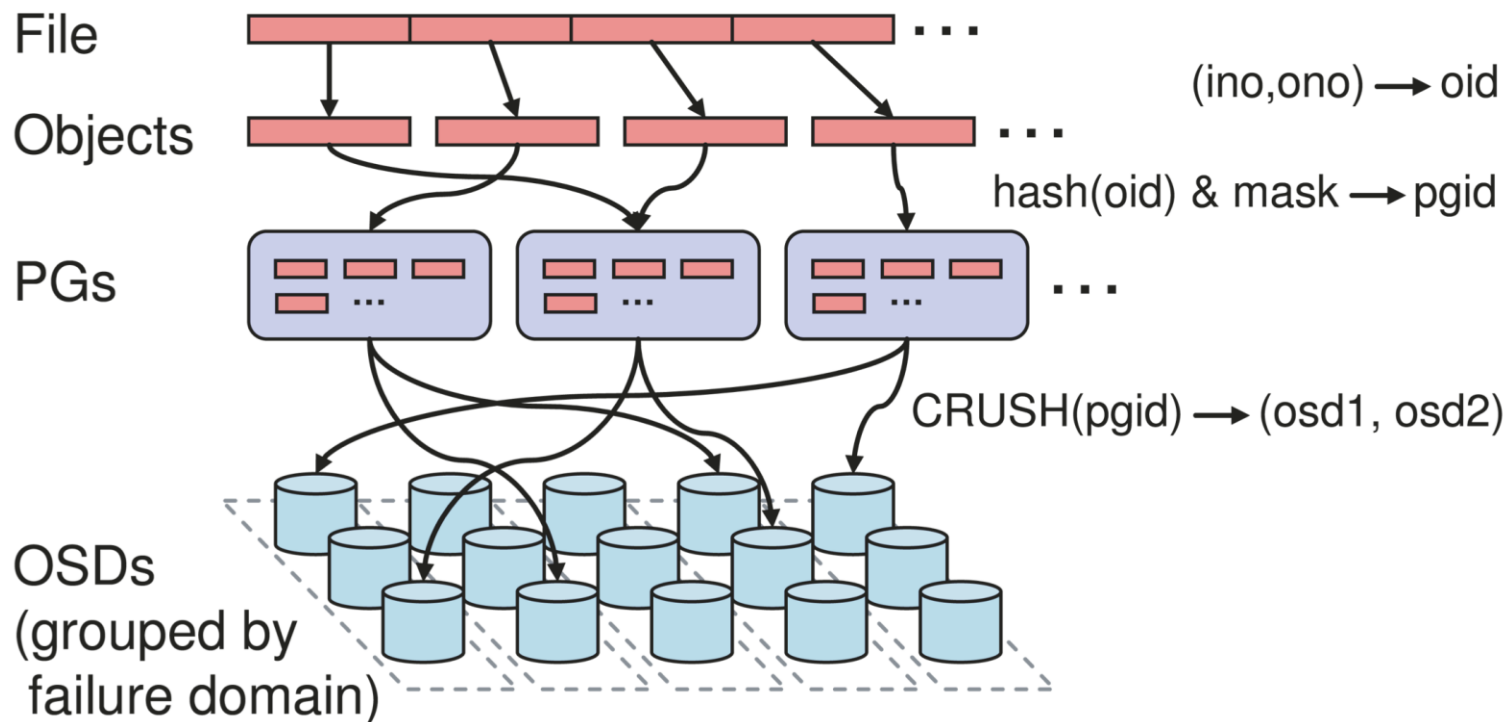
Ceph-CRUSH算法

- ❑ CRUSH是一个去中心化的算法，决定Ceph数据多副本的位置
- ❑ 通过一个树状结构来反映数据服务器集群的结构，包括机架、机器和存储设备等层次
- ❑ 每层通过类似一致性hash的算法来决定数据块的分配



Ceph-CRUSH算法

- ❑ Ceph中，将文件切分成多个数据块，多个数据块会存放在一个组中，称为PG，根据pgid来唯一索引
- ❑ $pgid = HASH(name) \bmod PG\ NUM$



Ceph-CRUSH算法

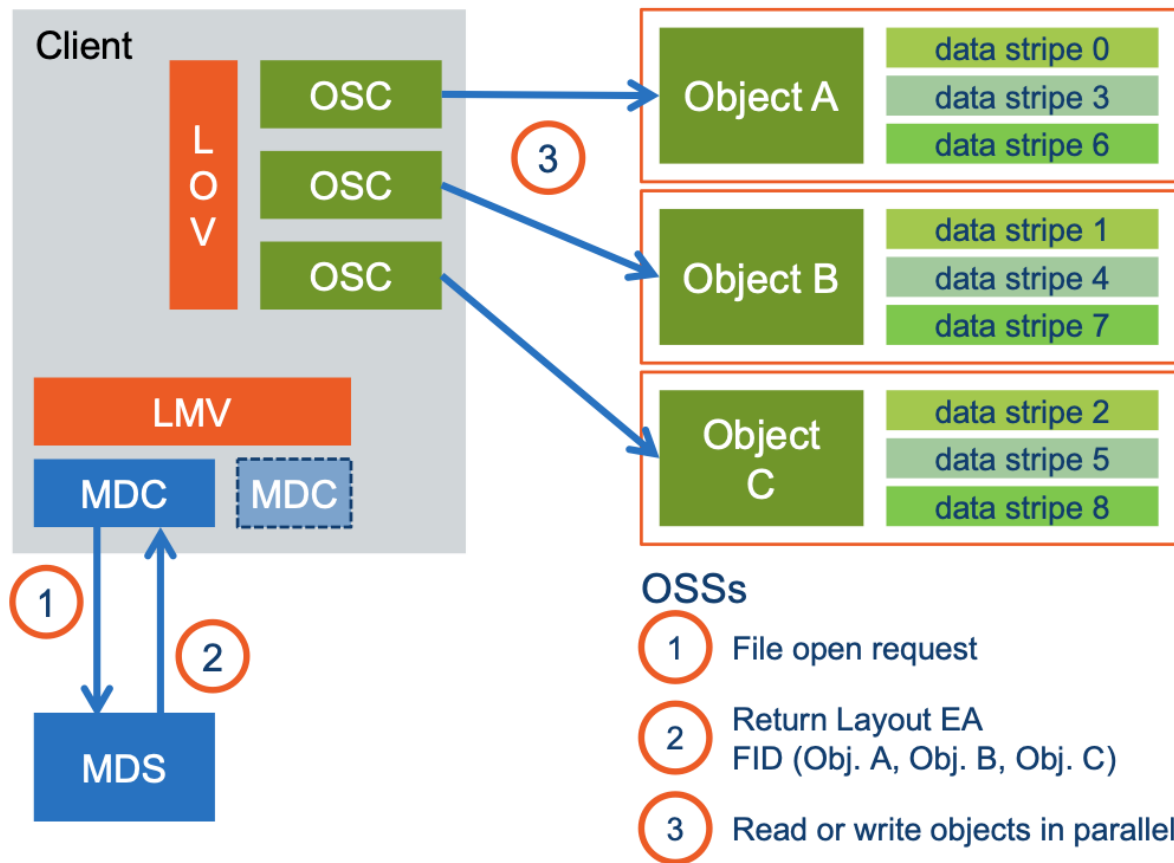
- ❑ PG会被映射到数据存储服务器
- ❑ 如何根据数据存储服务器的性能（容量）来决定数据分配？
- ❑ 解决：带权重的hash算法

$$C(pgid, \vec{i}, r) = \operatorname{argmax}_{i \in \vec{i}} \operatorname{HASH}(pgid, r, ID(i)) \times W(i),$$

- ❑ 将pgid和ID等信息通过哈希函数计算之后，再和每个预先分配的服务器权重相乘，取乘积最大的那台服务器作为数据块的存储
- ❑ 多副本反复执行上述算法，直到选出符合要求的n个数据副本存放位置

中心化数据块分布

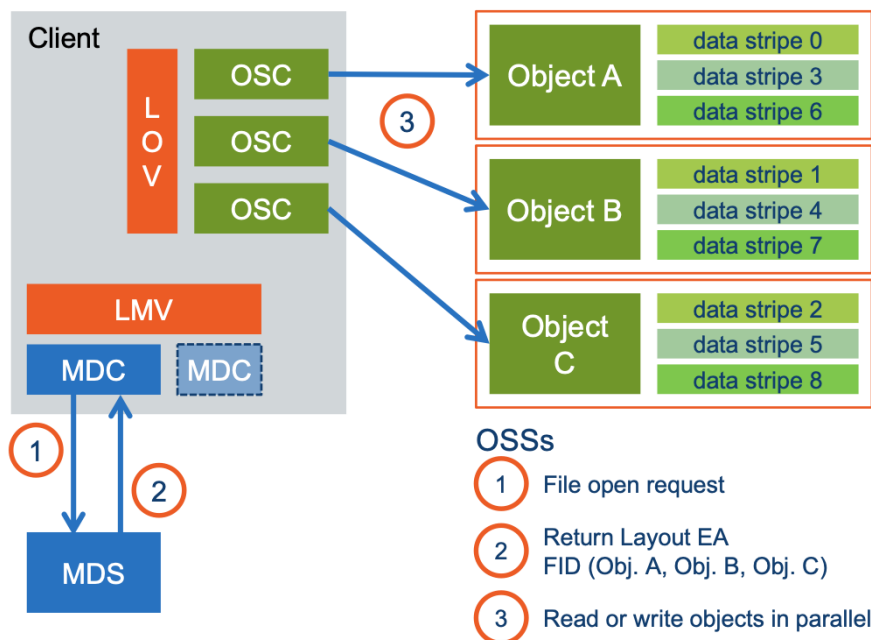
□ Lustre



中心化数据块分布

□ 读写流程和GFS类似

1. 客户端向元数据服务器请求数据服务器位置信息
2. 元数据服务器返回位置信息
3. 客户端直接和数据服务器通信

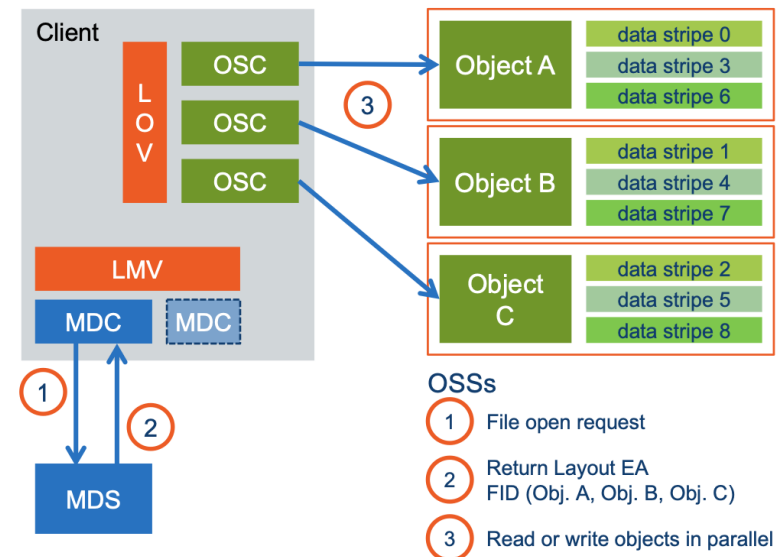


Lustre

❑ 元数据服务器决定数据块分布

❑ Round-robin allocator

- File 1: OST1, OST2, OST3, OST4
- File 2: OST5, OST6, OST7
- File 3: OST0, OST1, OST2, OST3, OST4, OST5
- File 4: OST6, OST7, OST0



目录

□ 元数据组织

- 元数据基本概念
- 元数据组织布局
- 路径解析

文件系统操作涉及的元数据访问

文件系统操作	涉及的元数据访问
Create/Delete	文件的 inode、父目录的时间戳
Mkdir/Rmdir	目录 inode、父目录时间戳
Open	路径上所有中间目录的权限和 entrylist、文件的权限
Read/Write	文件的时间戳
Append	文件的大小和时间戳
Readdir	目录 entrylist
Stat/Getattr/Setattr	文件/目录 inode
Rename	文件/目录 inode、两个父目录的时间戳和 entrylist

元数据访问十分密集

□ 研究表明，文件访问中超过 50% 的请求是针对元数据的访问^[1]

□ 例如，读文件 /foo/bar 的流程如下：

-
- The diagram illustrates the steps of reading a file /foo/bar. It consists of a list of seven steps. The first four steps are grouped under a purple bracket labeled 'open', and the last three steps are grouped under a purple bracket labeled 'read'.
- 1. 读 / 目录的 inode，检查访问权限
 - 2. 读 / 目录的 entrylist，查询子目录 foo 对应的 inode number
 - 3. 读 foo 目录的 inode，检查访问权限
 - 4. 读 foo 目录的 entrylist，查询文件 bar 对应的 inode number
 - 5. 读 bar 文件的 inode，检查访问权限，获取数据映射信息
 - 6. 读 bar 文件的数据
 - 7. 更新 bar 文件的 inode 中的访问时间戳

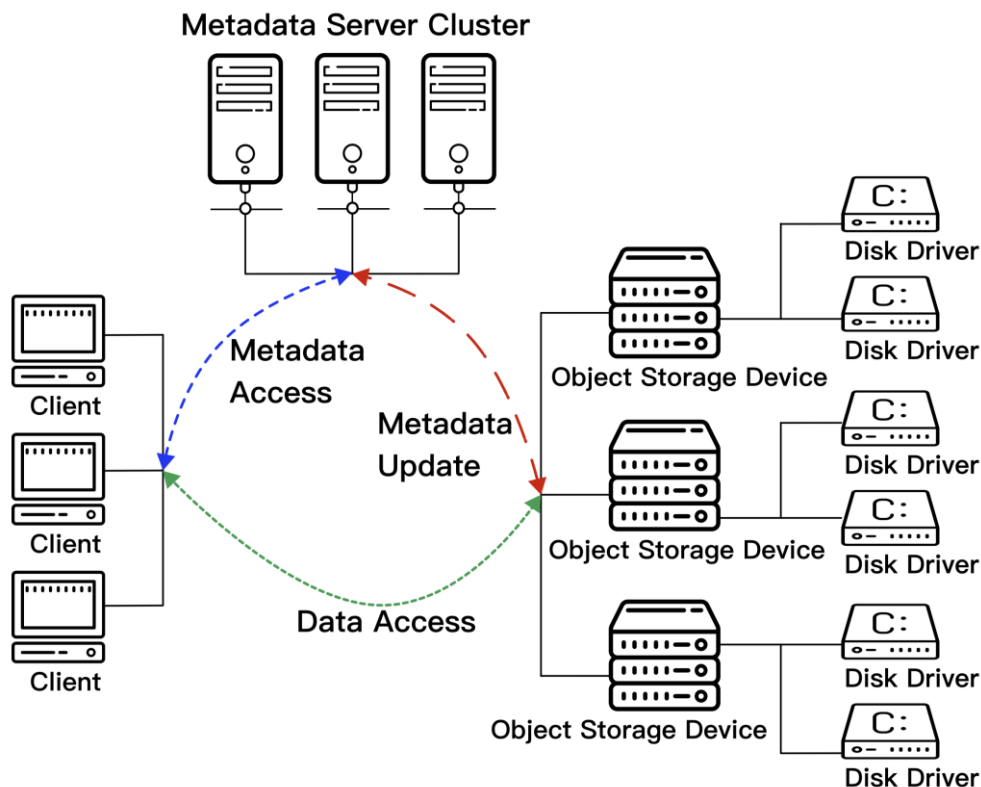
■ 其中，7次访问中有6次是针对元数据的访问，其中1-4步又被称为路径解析过程

[1] Measurement and analysis of large-scale network file system workloads. USENIX ATC, 2008.

分布式文件系统基本结构

□ 分布式文件系统通常采用元数据与数据解耦的管理方式

- 元数据由高性能元数据服务器集群单独管理
- 数据由对象存储集群进行管理



目录

□ 元数据组织

- 元数据基本概念
- 元数据组织布局
- 路径解析

元数据组织布局

□ 元数据的组织布局可以分为三大类：

1. 中心化元数据服务器架构

■ HDFS

2. 无元数据服务器架构

■ GlusterFS

3. 分布式元数据服务器架构

■ HDFS Federation、Lustre DNE、CephFS

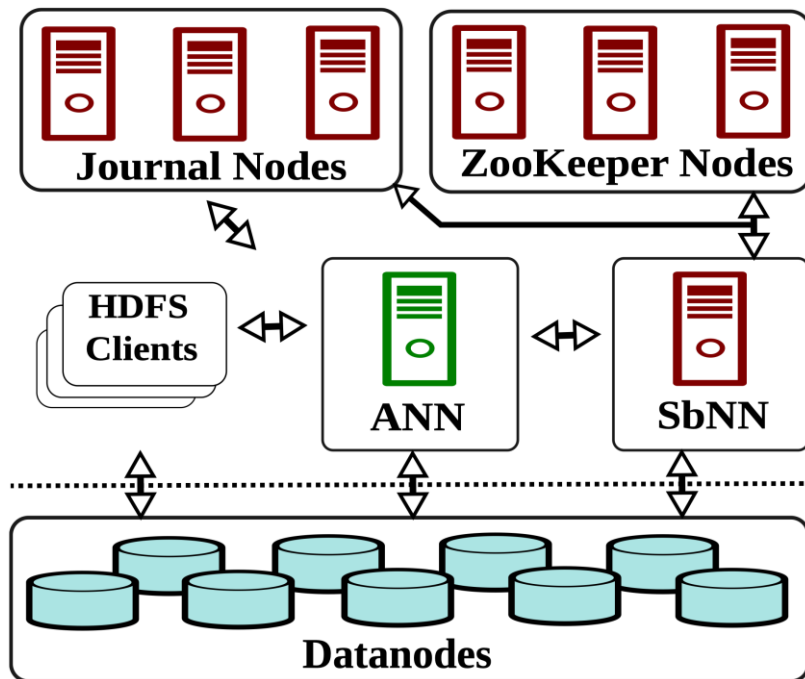
■ IndexFS、LocoFS、InfiniFS

1. 中心化元数据服务器架构 - HDFS

□ 这种架构采用单一中心化元数据服务器管理元数据信息，管理逻辑简单，易于维护；典型的有 HDFS，它是大数据处理框架 Apache Hadoop 的核心组件之一

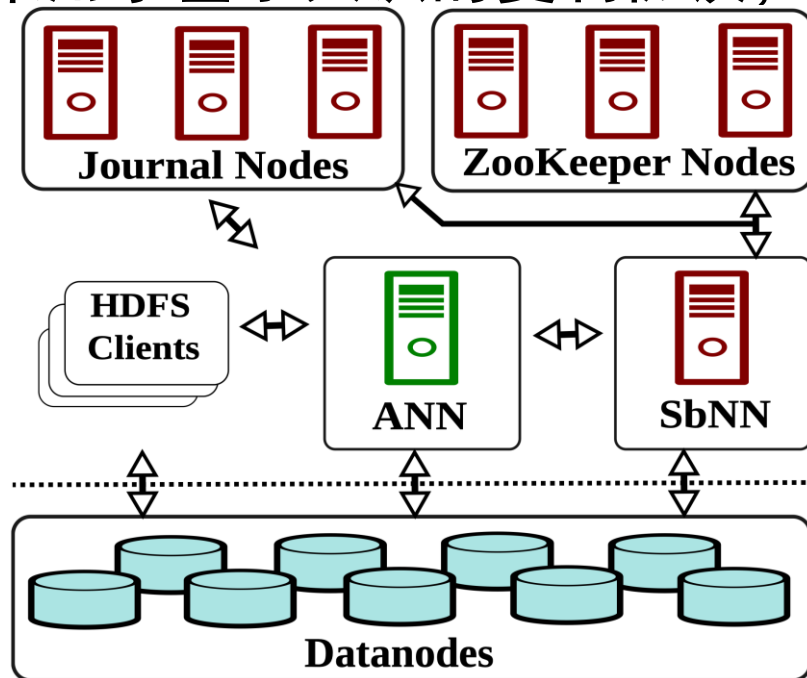
□ 如图，HDFS 元数据管理由三部分组成：

- 单一活跃元数据服务器（ANN，Active NameNode）
- 备用元数据服务器（SbNN，Standby NameNode）
- 日志服务器集群（Journal Nodes）



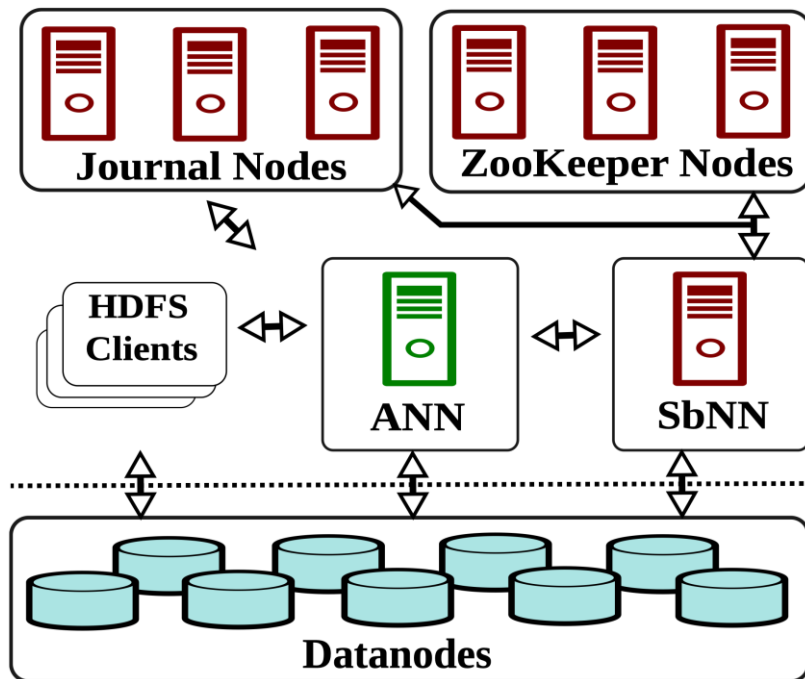
1. 中心化元数据服务器架构- HDFS

- ❑ 元数据全部存储在 Active NameNode (ANN) 的内存中，以保证元数据访问低延迟
- ❑ ANN 内部通过单一全局锁进行元数据访问的并发控制，实现了单写者多读者的并发语义
- ❑ ANN 会同步地将元数据更新发送到日志服务器集群 (Journal nodes) 中；日志集群采用了基于共识的复制协议，从而保证元数据信息的可靠性



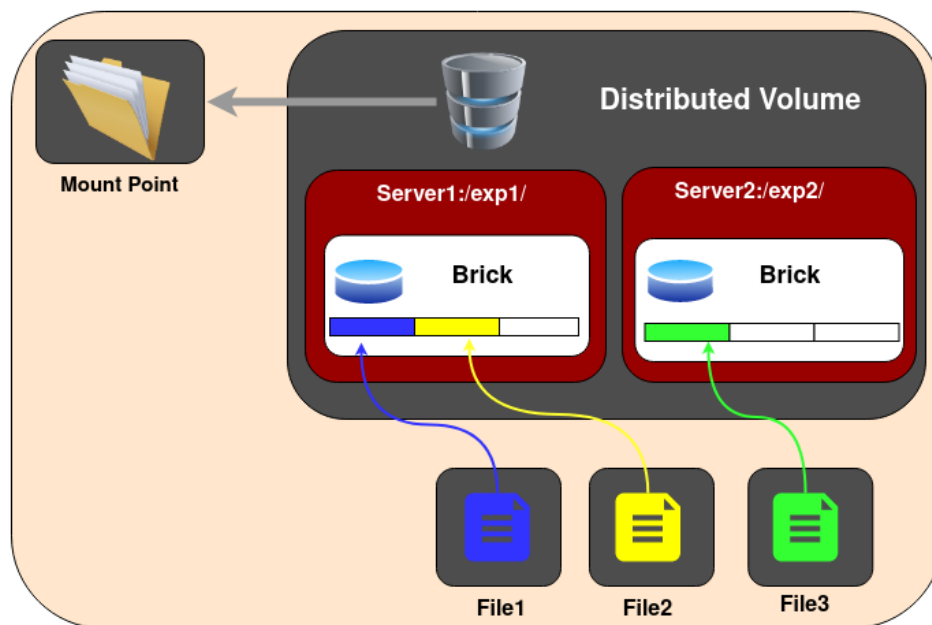
1. 中心化元数据服务器架构- HDFS

- 元数据更新会异步地更新到备用元数据服务器（SbNN）
- 当 ANN 节点崩溃时，集群会在 ZooKeeper 的协调下进行元数据服务器的故障切换，将 SbNN 提升为新的 ANN，从而保证分布文件系统服务的高可用性



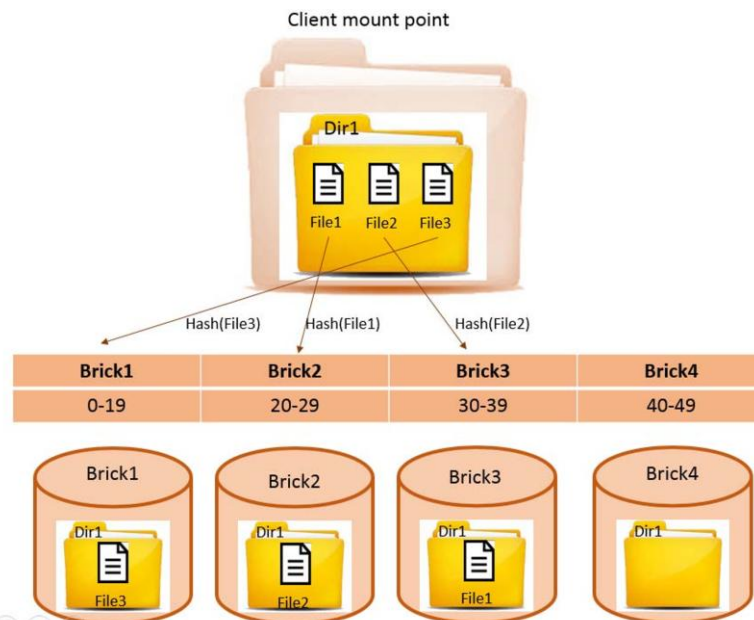
2.无元数据服务器架构- GlusterFS

- 这种架构去除了元数据服务器，将元数据和数据共同存储在数据服务器上，实现了无中心节点的分布式文件系统，典型的有 GlusterFS，它是一个优秀的开源分布式文件系统
- GlusterFS 的架构如下图所示
 - brick: 是存储节点提供的本地存储资源，例如 hostname:dirpath
 - volume: 是 GlusterFS 文件系统的存储单元，包含一个或多个 brick；客户端通过挂载 volume 来访问 GlusterFS 文件系统



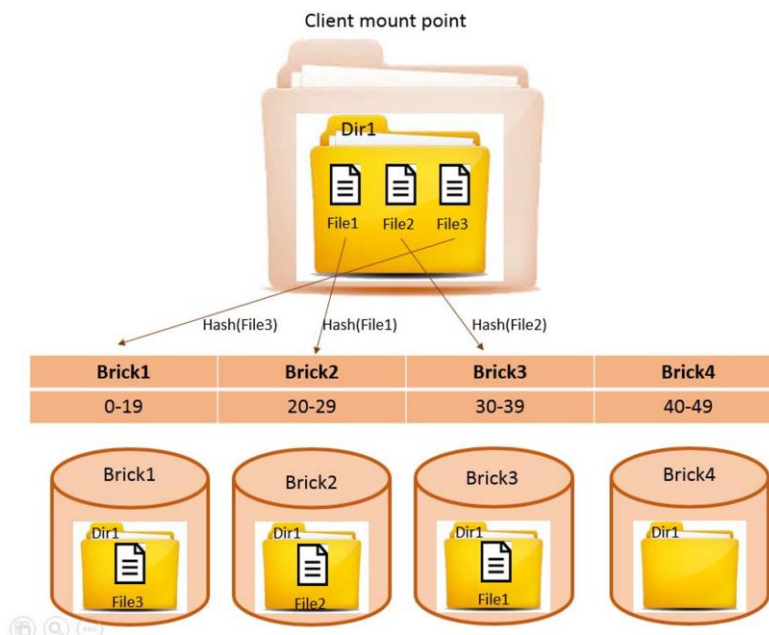
2.无元数据服务器架构- GlusterFS

- ❑ GlusterFS 将目录复制到所有 brick 上
- ❑ 创建目录时，GlusterFS 会向所有 brick 下发创建目录操作并分配目录哈希空间
 - 每个目录都有一个32位哈希空间，被分配到各个 brick 上，
 - 每个目录的哈希分布相互独立，分布信息记录在目录的扩展属性中，如此能够更均衡的保证节点之间容量的平衡



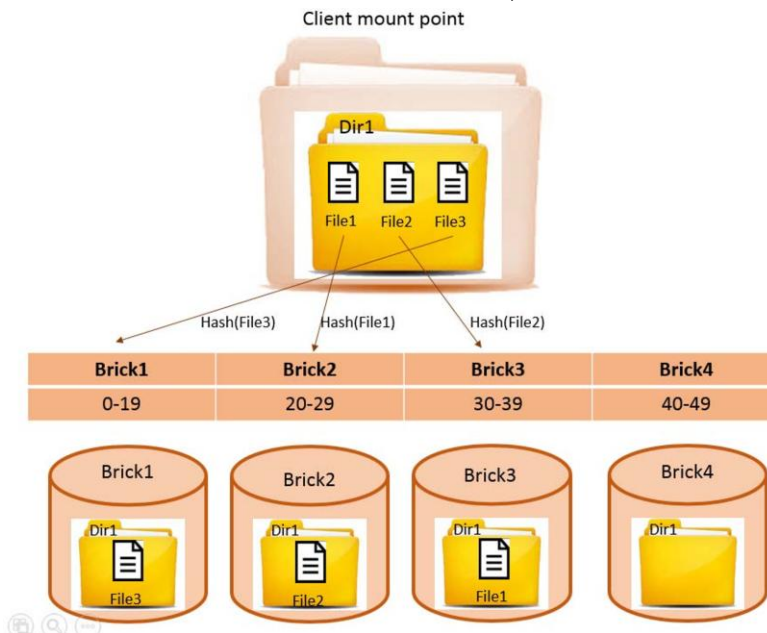
2.无元数据服务器架构- GlusterFS

- ❑ 与目录不同，文件及其元数据只存储在一个 brick 中
- ❑ 创建文件时，GlusterFS 需要先获取父目录的空间分配信息；然后计算文件名的哈希值，确定文件及其数据所在的 brick。例如，结合 $\text{Hash}(\text{File1}) = 22$ 与 Dir1 的哈希空间分配信息，文件存储在 Brick2 上



2.无元数据服务器架构- GlusterFS

- ❑ 对目录的操作需要广播到所有的 brick 上。
- ❑ 在路径解析过程中，查询目录时需要向 volume 中所有 brick 下发 lookup 操作，合并得到目录的哈希空间分配信息。例如，当客户端访问文件 /Dir1/File1 时（且缓存为空）：
 - 首先，客户端需要访问所有 brick 上的 Dir1 目录元数据，聚合得到目录的哈希空间分布信息，并检查目录访问权限
 - 然后，客户端计算文件名的哈希值，访问文件所在的 brick

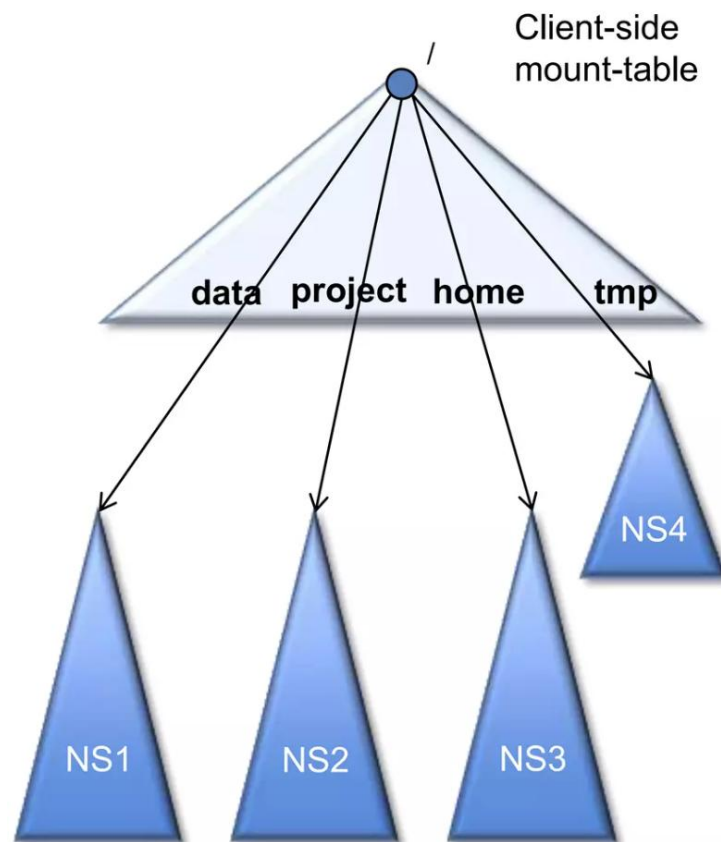


3.分布式元数据服务器架构

- 这种架构将元数据划分到多台元数据服务器上进行管理，以实现元数据容量和性能的横向扩展
- 根据对文件系统目录树的划分机制的不同，可以分为：
 1. 静态子树划分机制
 - HDFS Federation、Lustre DNE
 2. 动态子树划分机制
 - CephFS
 3. 访问-内容解耦的划分机制
 - InfiniFS (FAST' 22)

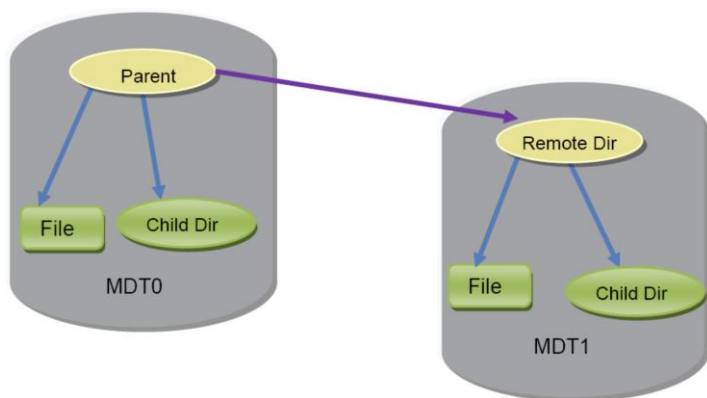
3.1 静态子树划分机制 – HDFS Federation

- ❑ HDFS Federation 是对 HDFS 重要的架构改进，解决了单一元数据服务器带来的性能瓶颈问题，提高扩展性
- ❑ 管理员需要手动划分目录树（即命名空间，NameSpace），并将每个子树分配给指定的元数据服务器
- ❑ 元数据服务器之间相互独立，不进行任何协调，不支持跨子树的操作
- ❑ 客户端通过挂载表（mount table）查询元数据所在服务器
- ❑ 客户端之间通过挂载同一子树来共享命名空间

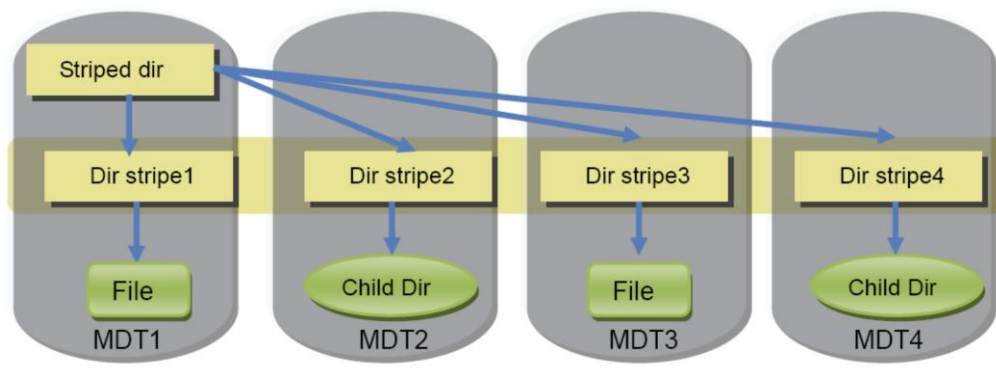


3.1 静态子树划分机制 – Lustre DNE

- ❑ Lustre 是一个广泛应用于高性能计算的开源分布式文件系统，于 v2.4 提出分布式命名空间（DNE）功能来支持元数据扩展
- ❑ Lustre DNE1：远程目录
 - 允许管理员手动将目录子树指派给某个元数据服务器管理
- ❑ Lustre DNE2：条带化目录
 - 条带化目录会自动将文件、子目录树散布到不同元数据服务器上



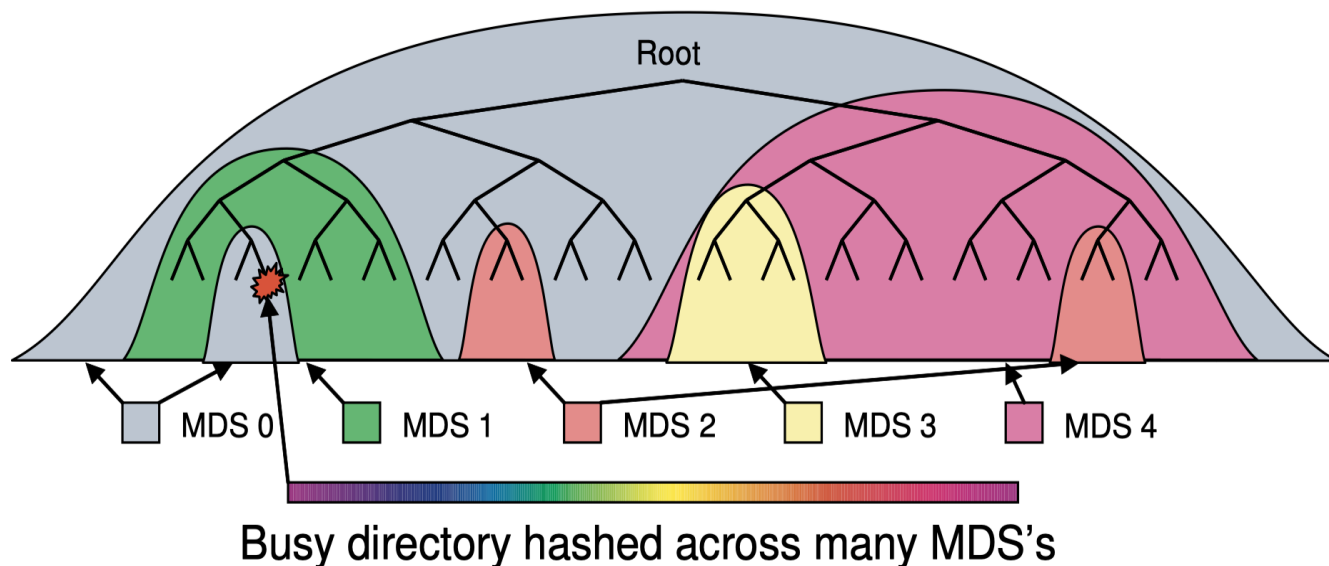
远程目录



条带化目录

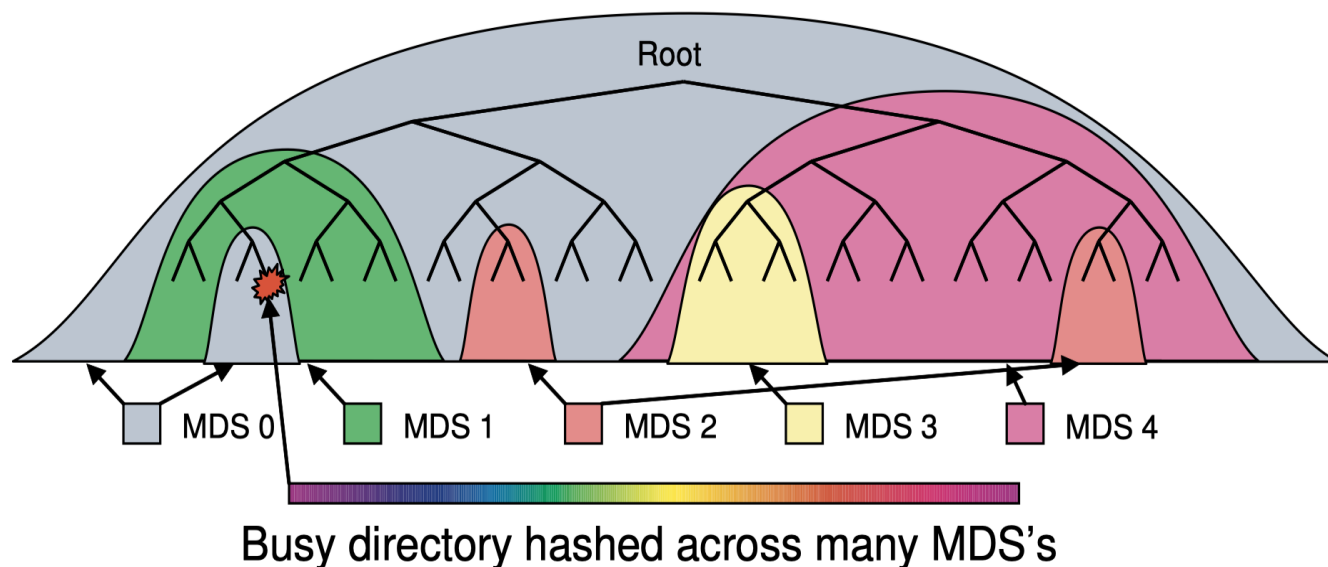
3.2 动态子树划分机制 – CephFS

- ❑ CephFS 是一个广泛使用的开源分布式文件系统，采用了动态子树划分来分割目录树，并在 workload 变换时迁移子树
- ❑ 目录树划分：每个子树都有唯一的一个权威 MDS。如图所示，MDS 0 负责灰色部分，MDS 1 负责绿色部分。热点子树可以被复制到多个 MDS 上，来服务元数据读请求，但元数据写请求只能转发给权威 MDS 进行处理。



3.2 动态子树划分机制 – CephFS

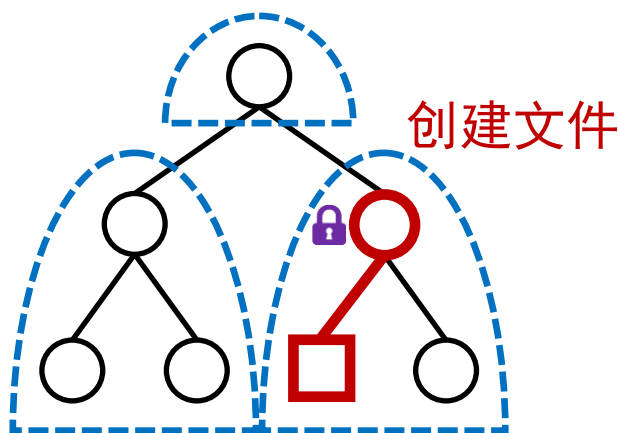
❑ **热点检测：** MDS 会收集本地的访问信息。每隔 10 秒，每个 MDS 会将本地的访问信息打包发送给集群中的其他 MDS。然后，每个 MDS 本地会聚合得到集群整体的访问信息。接着，每个 MDS 上的平衡器（balancer）独立判断是否需要迁移本地子树来平衡负载。最后，平衡器通过计数器识别本地热点子树，将其进行分割，并通过两阶段提交的方式将其迁移到其他 MDS 上。



3.3 访问-内容解耦的划分机制 – InfiniFS

问题：现有划分机制难以高效地支持千亿量级目录树

- 子树粒度划分机制：局部性好，但容易导致**负载不均**
- 细粒度划分机制：负载均衡，但元数据访问的**局部性差**

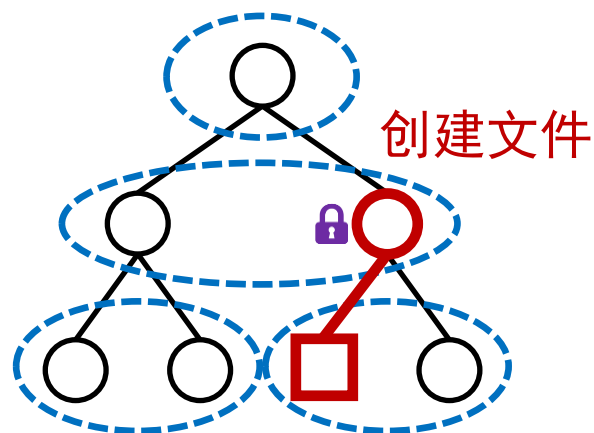


粗粒度划分

元数据局部性



负载均衡



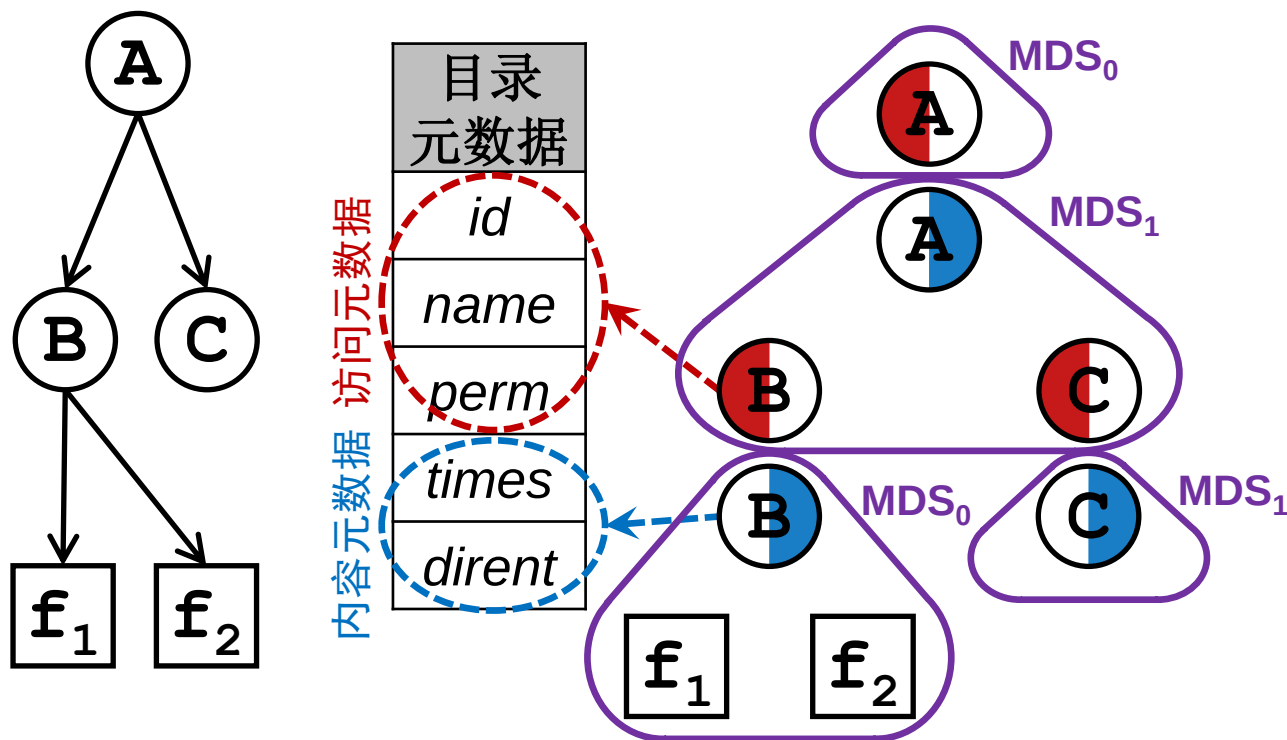
细粒度划分



3.3 访问-内容解耦的划分机制 – InfiniFS

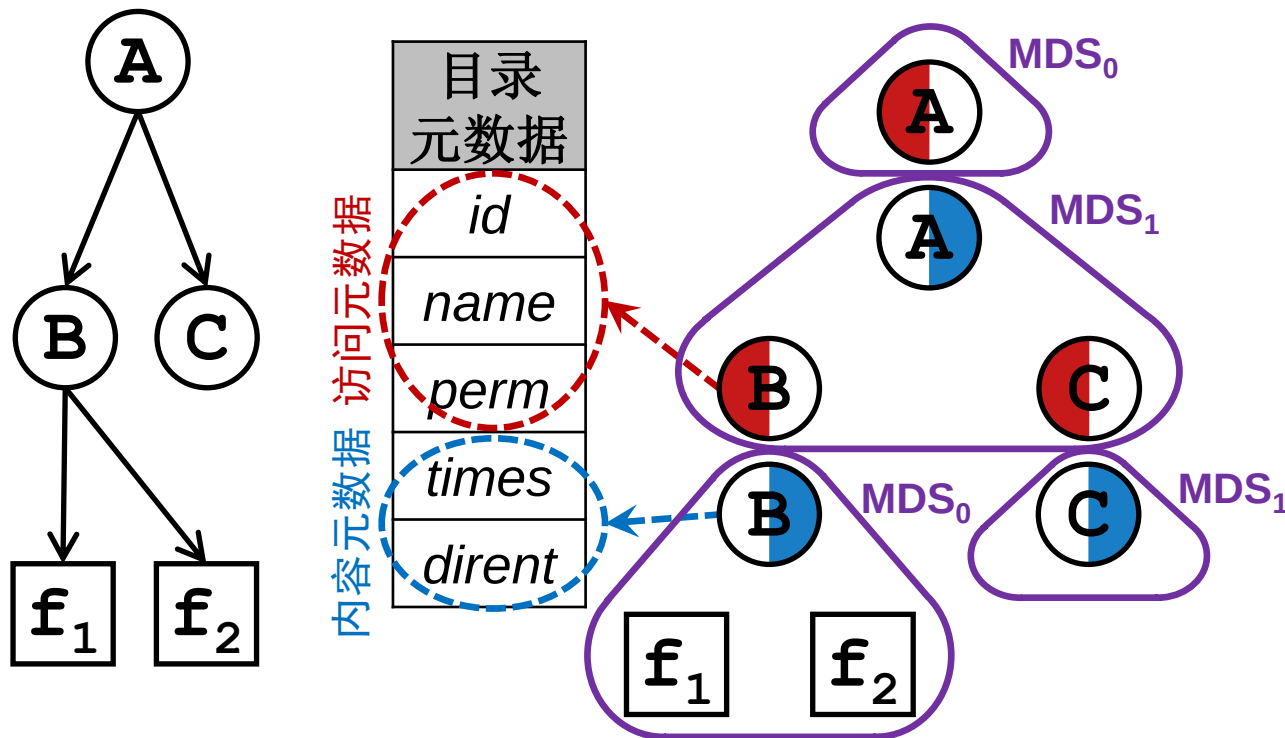
■ InfiniFS (FAST '22) 将目录元数据解耦合

- 与目录树访问相关的访问元数据
- 与孩子节点相关的内容元数据



3.3 访问-内容解耦的划分机制 – InfiniFS

- 然后，依据依赖关系组合元数据，保证元数据操作的局部性
 - 将访问元数据与父节点绑定
 - 将内容元数据与孩子节点绑定
 - 如图，创建 /A/B/f3 只在 MDS0 上进行，无需分布式协议
- 最后，通过哈希散布元数据组合，保证元数据的负载均衡



目录

□ 元数据组织

- 元数据基本概念
- 元数据组织布局
- 路径解析

路径解析

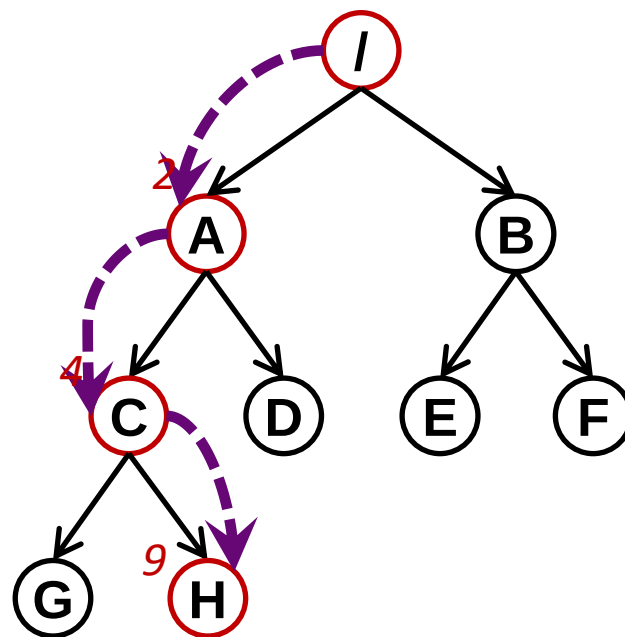
□ 路径解析位于文件访问的关键路径上，指的是将请求的路径索引到目标文件元数据的过程。典型的元数据索引方法可以分为三类：

1. 层级索引方法
2. 全路径索引方法
3. 基于预测执行的并行解析方法

AI对大规模数据集存储的需求

□ 海量小文件难管理：海量多模态音视频、图片样本

- 元数据路径解析延迟高
- 需要更高效的元数据管理机制



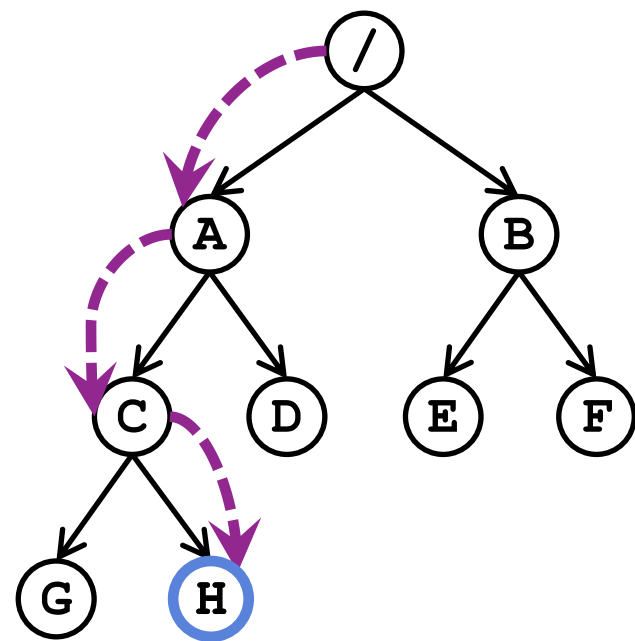
路径解析延迟 $\propto$ 文件规模

1. 层级索引方法

- ❑ 由于文件系统通过目录树的方式组织文件和目录，并通过权限的方式进行数据隔离与共享，路径解析过程通常需要从根目录开始一层层地索引下一级目录，并检查目录权限
- ❑ Lustre, CephFS, IndexFS 等

open(/A/C/H)

1. 根据 / 目录的元数据，检查访问权限，并查询子目录 A 的元数据位置信息
2. 根据 A 目录的元数据，检查访问权限，并查询子目录 C 的元数据位置信息
3. 根据 C 目录的元数据，检查访问权限，并查询子文件 H 的元数据位置信息
4. 打开文件，返回文件元数据

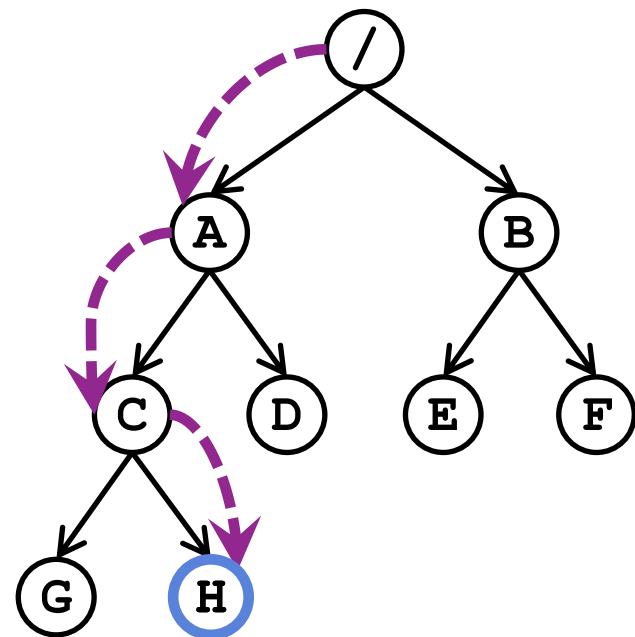


2. 全路径索引方法

- ❑ 层级索引方法在分布式元数据架构下会带来多次串行网络访问，导致元数据访问延迟高
- ❑ CalvinFS (FAST '15) 提出全路径索引方法来加速路径解析
 1. 使用文件/目录的全路径作为其元数据的 key，元数据存储于键值存储系统中；
 2. 每个文件/目录元数据会存储所有祖先目录（直到根目录）的权限信息；

open(/A/C/H)

1. 以路径名 /A/C/H 作为 key，找到文件 H
2. 根据文件 H 元数据中存储的 /、A、C 的权限信息，进行权限检查
3. 打开文件，返回文件元数据



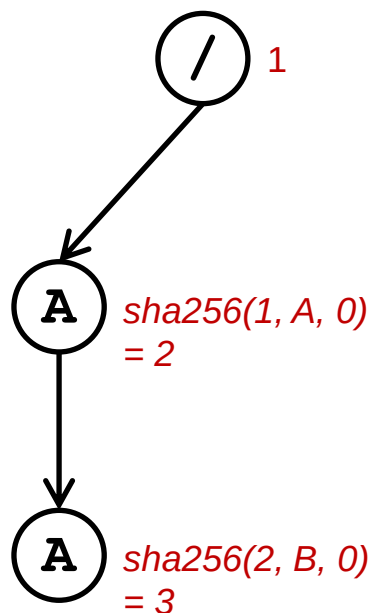
3. 基于预测执行的并行解析方法

- ❑ 全路径索引方法能将路径解析延迟降低到 1 RTT，然而它会导致重命名与权限修改操作需要更新整个子树下的所有目录和文件，当元数据量较大时将难以接受
- ❑ InfiniFS (FAST '22) 提出了基于预测执行的并行解析方法，它能够有效降低路径解析延迟到 1 RTT，同时不引入上述问题
- ❑ 基于预测执行的并行解析方法分为两大部分：
 - 可预测的目录 ID
 - 并行路径解析

3. 基于预测执行的并行解析方法

可预测的目录 ID

- ❑ 目录的 ID 在创建时通过哈希函数生成：SHA256(父目录 ID, 目录名, 版本号)
- ❑ 版本号默认为 0，除非检测到 ID 碰撞



3. 基于预测执行的并行解析方法

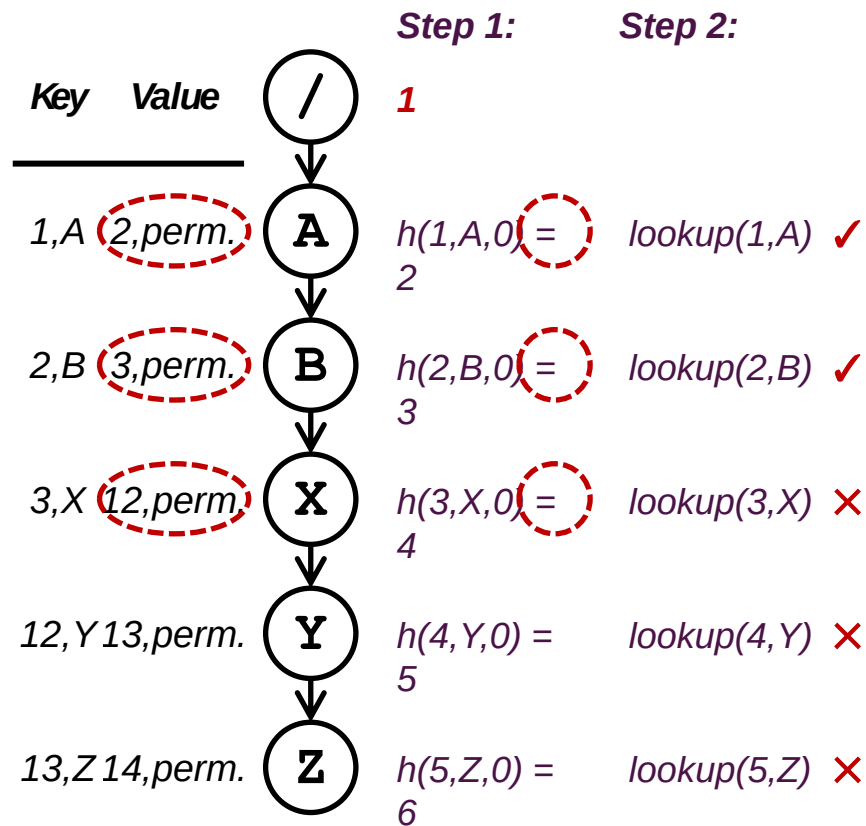
并行路径解析

步骤 1. 预测中间目录的 IDs

步骤 2. 并行执行 lookup

❖ 检查访问权限

❖ 验证预测的 ID



3. 基于预测执行的并行解析方法

并行路径解析

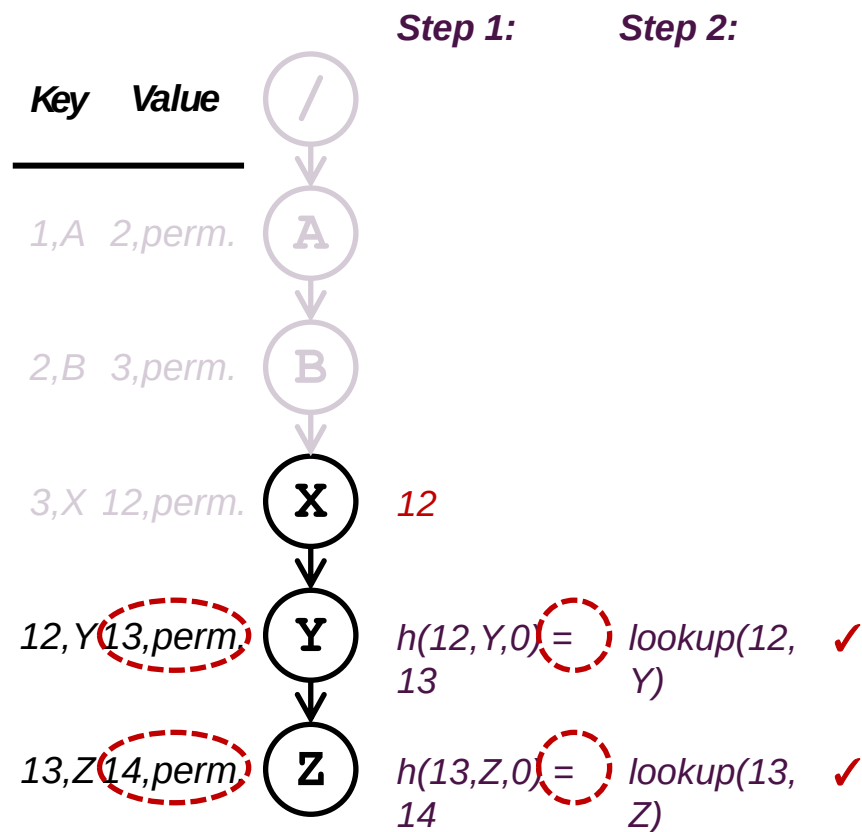
步骤 1. 预测中间目录的 IDs

步骤 2. 并行执行 lookup

❖ 检查访问权限

❖ 验证预测的 ID

步骤 3. 重复步骤 1-2 直到完成



3. 基于预测执行的并行解析方法

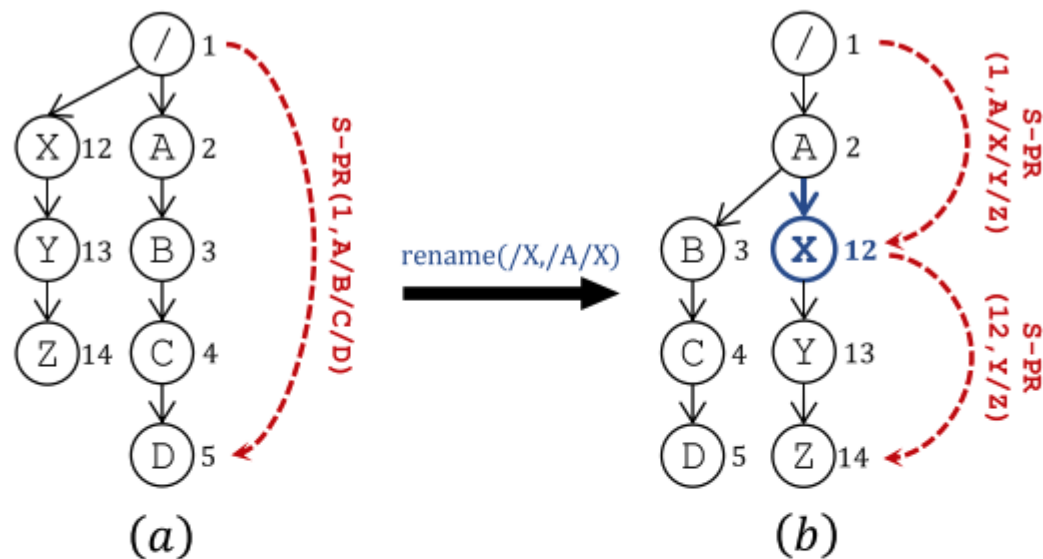


Figure 5: (a) Speculative path resolution (S-PR) reduces the latency of path resolution to nearly one network round-trip time, if correctly predicted. (b) After renaming the directory `/X` to be a child of `A`, resolving `/A/X/Y/Z` requires two rounds, taking about two network round-trip times.